

UNIT - II

3

PL / SQL

Syllabus

Concept of Stored Procedures and Functions, Cursors, Triggers, Assertions, Roles and Privileges.

Contents

3.1	Basics of PL/SQL	
3.2	Writing First PL/SQL Script	
3.3	Block Structure of PL/SQL	
3.4	PL/SQL Data Types	
3.5	PL/SQL Variables	
3.6	PL/SQL Constants	
3.7	Control Statements	
3.8	Handling Database Tables using PL/SQL	
3.9	Examples on PL/SQL.....	Aug.-17, Oct.-18,19, May-19,Dec.-19, Marks 5
3.10	Concept of Stored Procedures	
3.11	Functions	Aug.-17, Marks 5
3.12	Cursors	Dec.-17, Marks 5
3.13	Triggers	Dec.-17, 18, 19, Marks 5
3.14	Assertions	
3.15	Roles and Privileges	
3.16	Exceptions	Dec.-18, May-19, Marks 5
Multiple Choice Questions		

3.1 Basics of PL/SQL

- PL/SQL stands for Procedural Language extensions to the Structured Query Language (SQL).
- PL/SQL is a combination of SQL along with the procedural features of programming languages.
- **Oracle** uses a PL/SQL engine to process the PL/SQL statements.
- PL/SQL includes procedural language elements like conditions and loops. It allows declaration of constants and variables, procedures and functions, types and variable of those types and triggers.

3.1.1 How to Set Environment for Executing PL/SQL Scripts ?

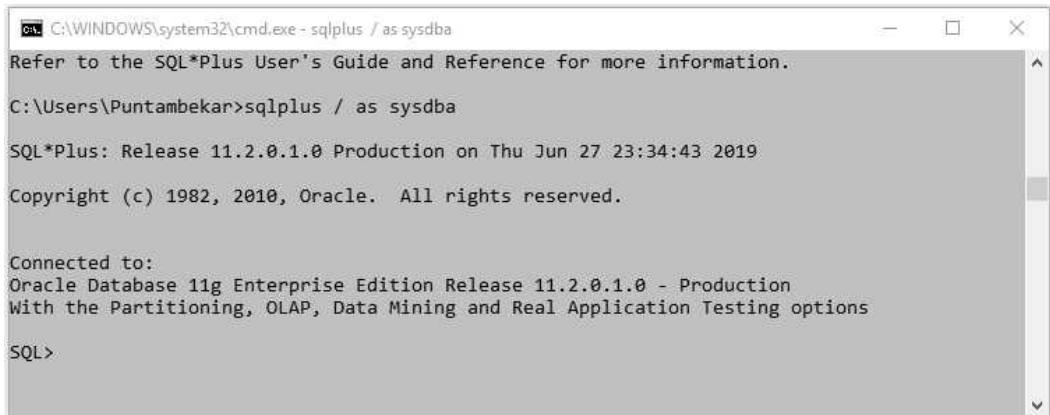
We need to install **Oracle** for executing the PL/SQL script. We can install it on Linux as well as on Windows (32 bit or 64 bit) platform.

For that purpose go to the web site <https://www.oracle.com>.

On completion of installation, Open command prompt window and issue the following commands at the prompt

```
Set ORACLE_SID=orcl
sqlplus /as sysdba
```

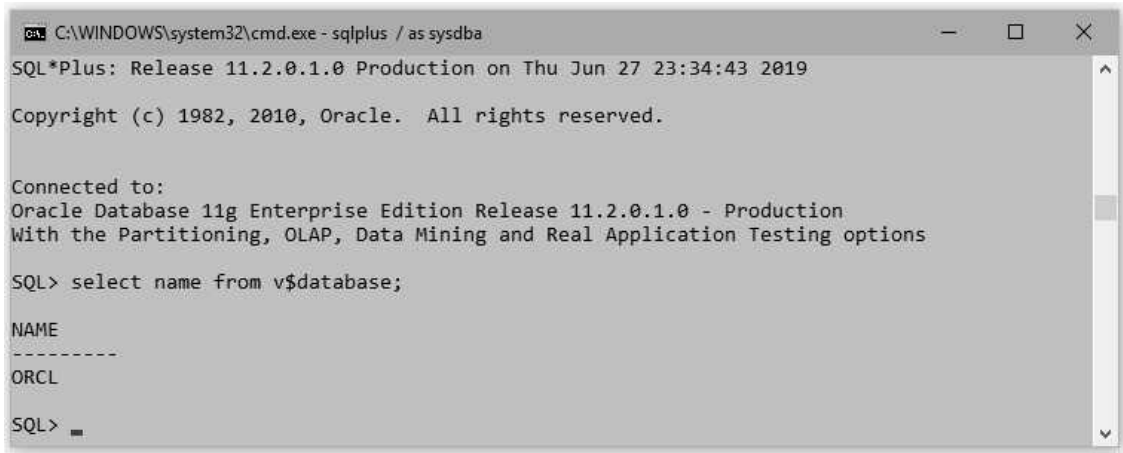
The command prompt window will display SQL prompt as follows -



```
C:\WINDOWS\system32\cmd.exe - sqlplus / as sysdba
Refer to the SQL*Plus User's Guide and Reference for more information.
C:\Users\Puntambekar>sqlplus / as sysdba
SQL*Plus: Release 11.2.0.1.0 Production on Thu Jun 27 23:34:43 2019
Copyright (c) 1982, 2010, Oracle. All rights reserved.

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
SQL>
```

You can test the Oracle installation by typing the SELECT query as follows.



```
C:\WINDOWS\system32\cmd.exe - sqlplus / as sysdba
SQL*Plus: Release 11.2.0.1.0 Production on Thu Jun 27 23:34:43 2019
Copyright (c) 1982, 2010, Oracle. All rights reserved.

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> select name from v$database;

NAME
-----
ORCL

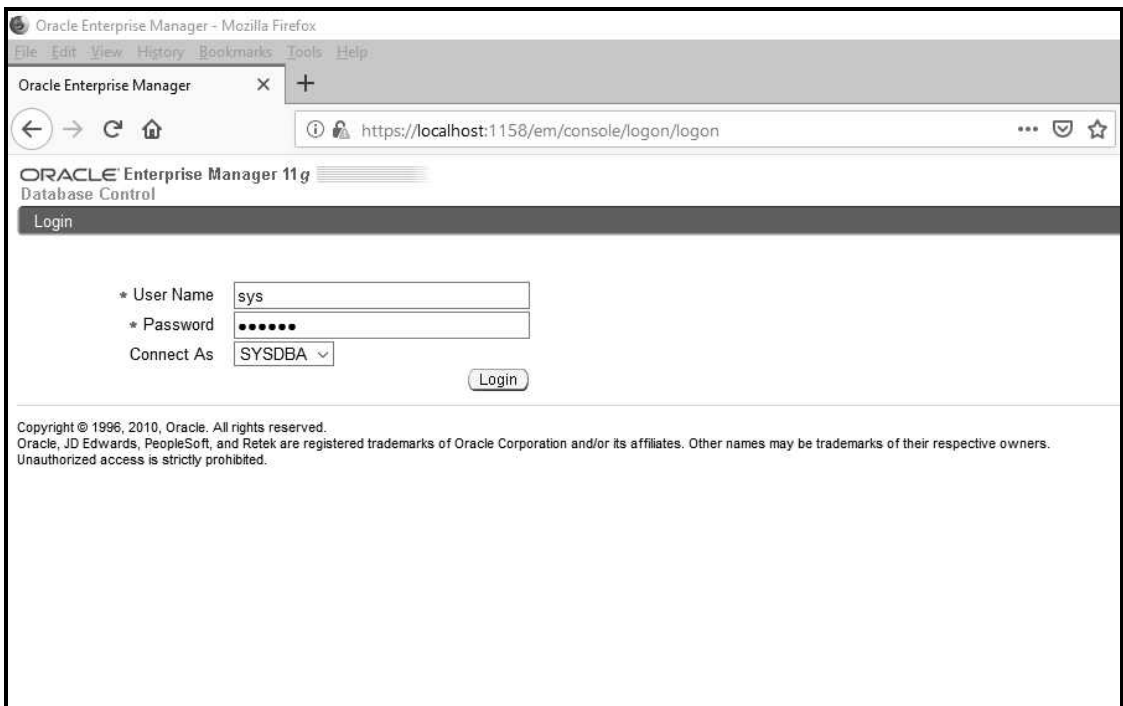
SQL> _
```

Then type **quit** and then exit at the SQL prompt to close this command prompt window. We can also check the installation is correct or not at GUI level as follows -

Just open the Web browser and type the URL as,

<https://localhost:1158/em>

Following type of web page appears in your browser.

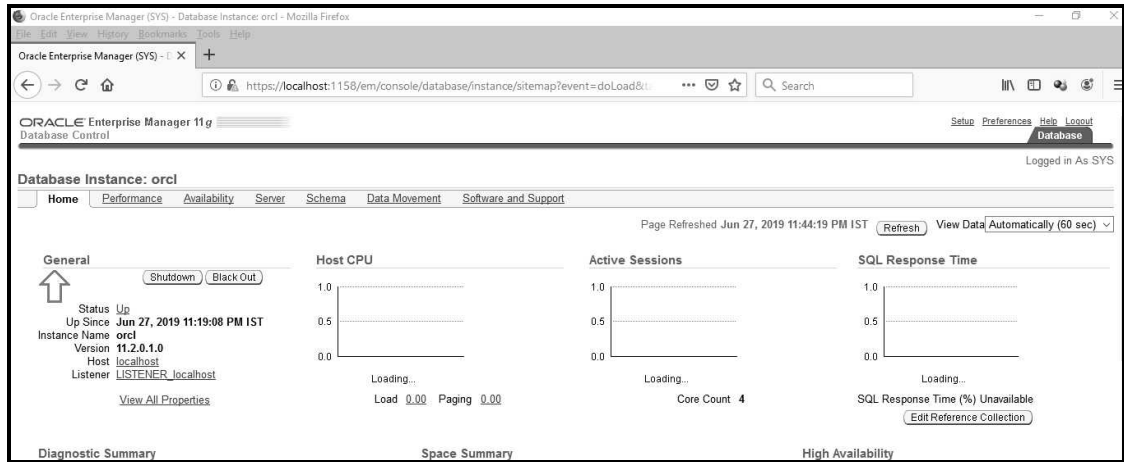


Username as **sys**

Password as **oracle**(you can choose any password of your choice)

Connect as SYSDBA

Click Login button and then the Web page appears as follows -



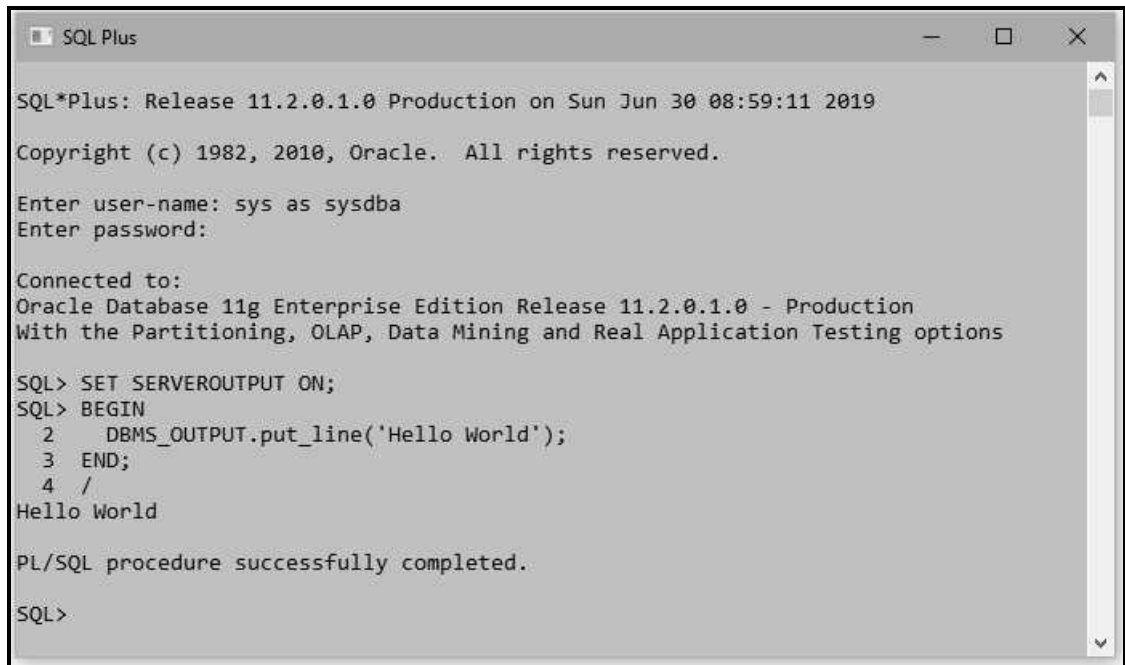
That it!! Your Oracle is installed on your machine.

Troubleshooting : Sometimes at the start of installation itself, it gives error about CREATE file on Command prompt, to resolve this error, just disable your Antivirus Software during the installation process.

3.2 Writing First PL/SQL Script

(1) Writing and Executing the PL/SQL script on SQL PLUS

- SQL Plus, which is a command-line interface for executing SQL statement and PL/SQL blocks provided by Oracle Database.
- Open SQL PLUS command prompt.
- Type user name and password. I am using the user name : sys as sysdba and password as oracle(This is the password which I set during installation.).
- Then the SQL prompt appears. Now we can execute our PL/SQL code from here.



```
SQL*Plus: Release 11.2.0.1.0 Production on Sun Jun 30 08:59:11 2019

Copyright (c) 1982, 2010, Oracle. All rights reserved.

Enter user-name: sys as sysdba
Enter password:

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> SET SERVEROUTPUT ON;
SQL> BEGIN
  2  DBMS_OUTPUT.put_line('Hello World');
  3  END;
  4  /
Hello World

PL/SQL procedure successfully completed.

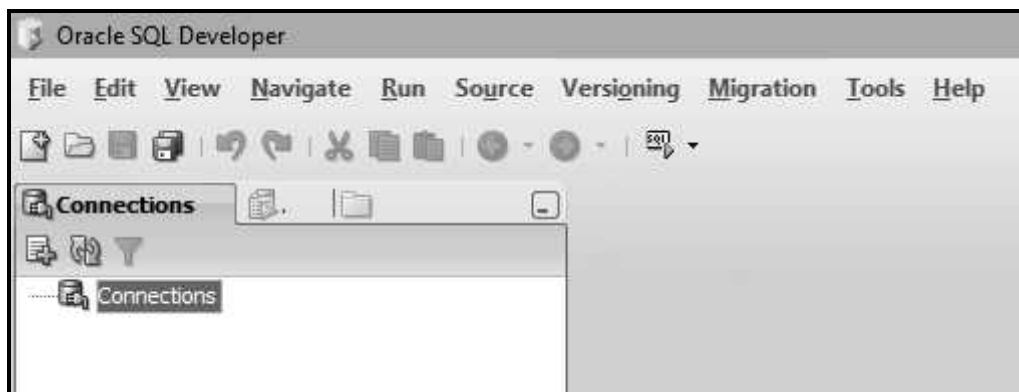
SQL>
```

(2) Writing the PL/SQL script on Oracle SQL Developer

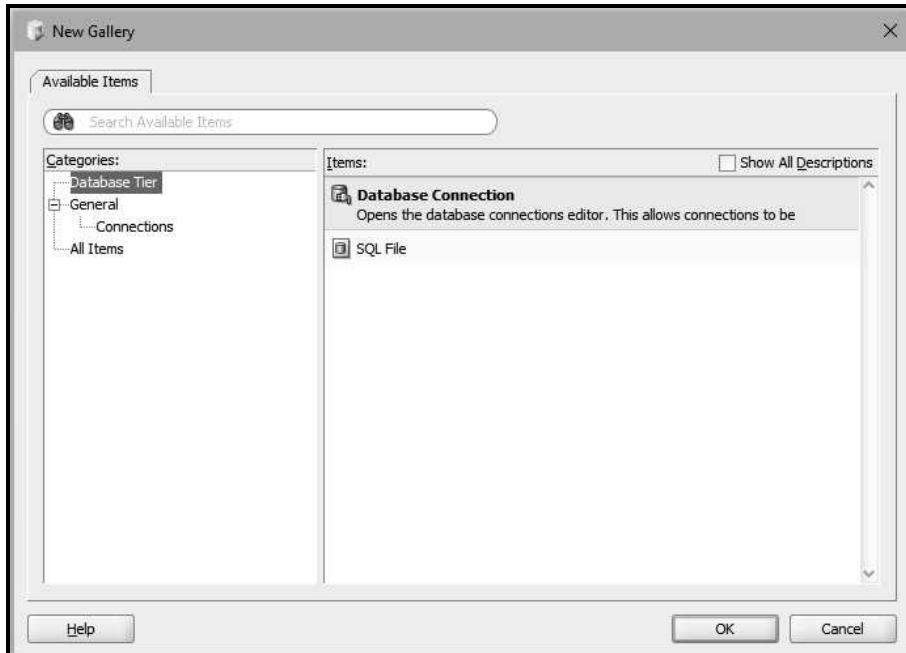
You can write PL/SQL script at the SQL Prompt or you can store the script in separate file and execute it.

For storing the script in separate file we need to use **Oracle SQL Developer**.

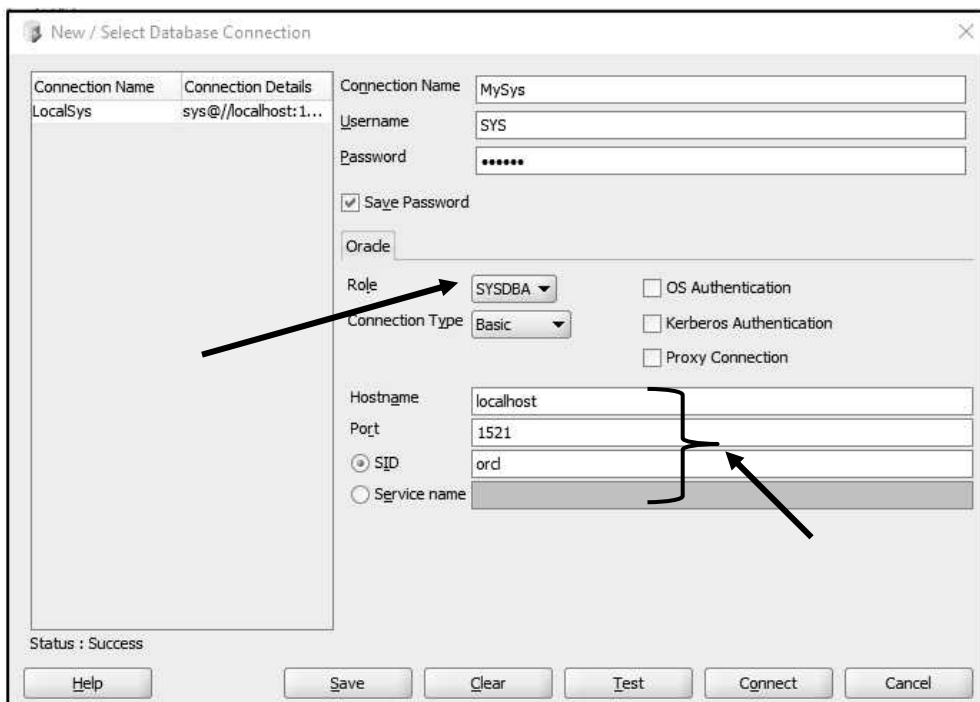
After installation of Oracle Package, this developer tool comes integrated with it. Click on it.



Click on File -> New, Following window will appear

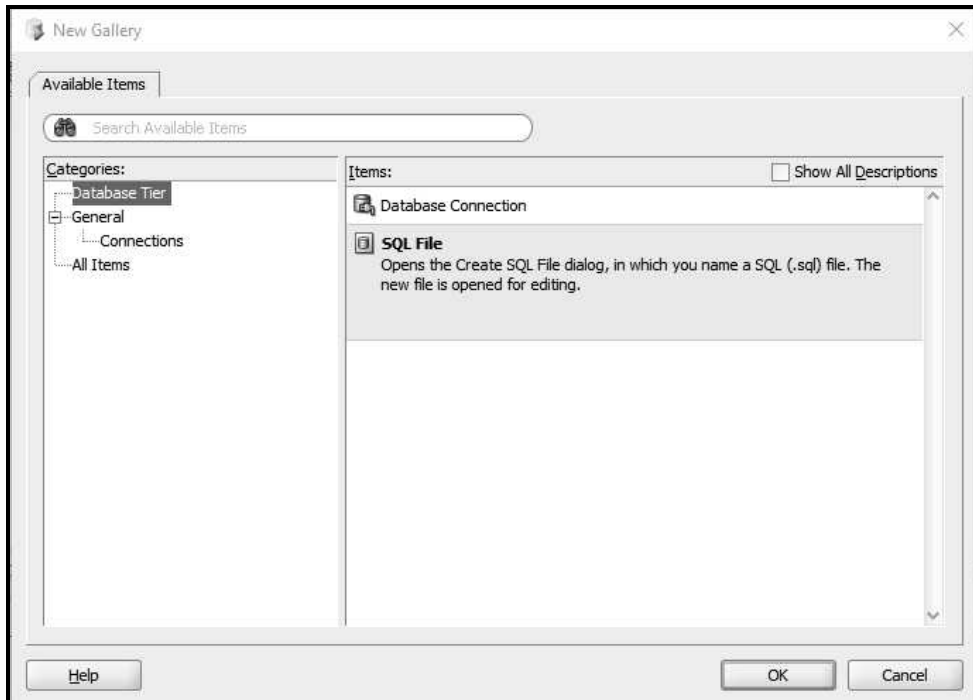


Now Enter user name and password as follows. You can give Connection Name of your choice. Then click Test button. On successful connection we get Success message. Then Click Connect button.

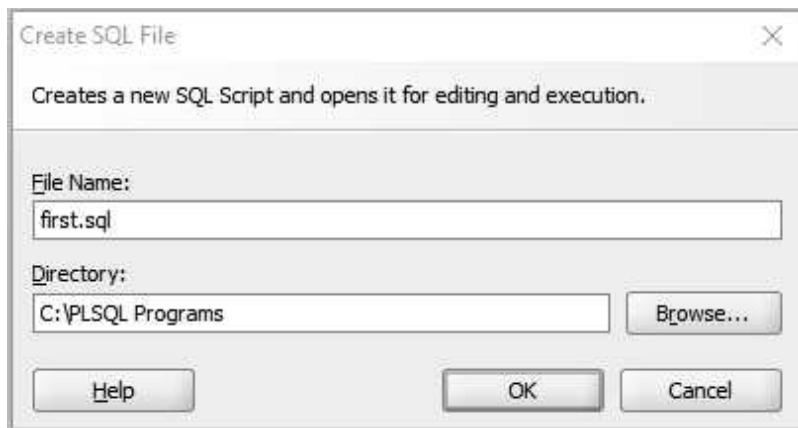


Now Click on **File->New**.

Following window appears. Click on **SQL File** and then Click **OK** button



Set the filename of your choice. For instance - I have created a PL/SQL file as follows -



The script is as follows -

```
SET SERVEROUTPUT ON;  
BEGIN  
    DBMS_OUTPUT.put_line('Hello World');  
END;  
/
```

Explanation of Script

(1) The first line of PL/SQL script is

```
SET SERVEROUTPUT ON;
```

This allows the user to see the output, when the script is executed on specified connection.

(2) Then BEGIN statement indicated the beginning of the execution block.

(3) The execution block is within the BEGIN and END statements. You can write any executable PL/SQL statements here.

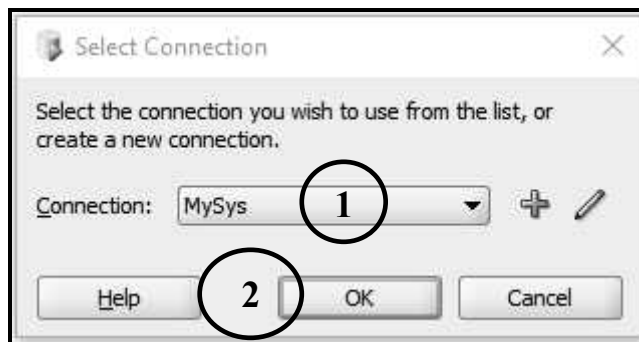
(4) For displaying the desired message we use DBMS_OUTPUT.PUT_LINE. The DBMS_OUTPUT is a built-in package that allows us to display output.

Execution of Script

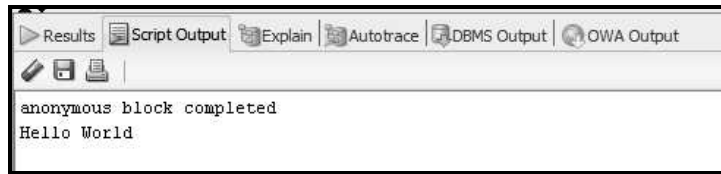
Click on Execution Script



The window for Select Connection will appear, Select the Connection and click OK button.



The output will be displayed at the bottom of the Editor window as follows



3.3 Block Structure of PL/SQL

- PL/SQL is a procedural language.
- The code is written in small blocks in PL/SQL.
- Broadly there are two types of blocks -

(1) Anonymous Block

- A block without a name is an anonymous block.
- An anonymous block is not saved in the Oracle Database server, so it is just for one-time use.
- PL/SQL anonymous blocks are useful for testing purposes.

(2) Named Block

- A PL/SQL block has a name.
- A **Function** or a **Procedure** is an example of a named block.
- A named block is saved into the Oracle Database server first and then can be reused.
- A PL/SQL block consists of three sections : declaration, executable, and exception-handling sections. In a block, the executable section is mandatory while the declaration and exception-handling sections are optional. This structure of PL/SQL block is as follows

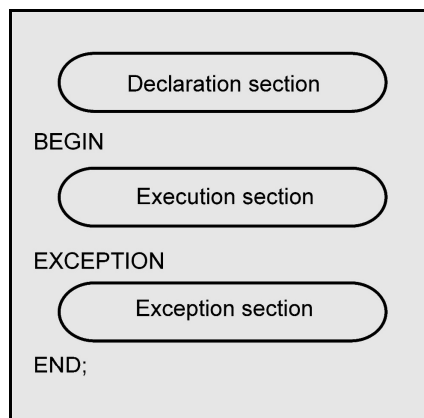


Fig. 3.3.1 Block structure of PL/SQL

1) Declaration section

A PL/SQL block has a declaration section where you declare variables, allocate memory for cursors and define data types.

2) Executable section

A PL/SQL block has an executable section. An executable section starts with the keyword BEGIN and ends with the keyword END. The executable section must have at least one executable statement, even if it is the NULL statement which does nothing.

3) Exception-handling section

A PL/SQL block has an exception-handling section that starts with the keyword EXCEPTION. The exception-handling section is where you catch and handle exceptions raised by the code in the execution section.

3.4 PL/SQL Data Types

Following are the types of PL/SQL data types

1. **Scalar** : These data types don't include any internal components. It includes data types such as NUMBER, DATE, BOOLEAN, etc.
2. **Large Objects (LOB)** : This type of data type stores objects that are relatively large in size and stored separately from other data types such as text, graphic images, video clips, sound, etc.
3. **Composite** : These type of data types have internal components that can be accessed individually. It includes records and collections.
4. **Reference** : As the name sounds, it includes pointers that refer to the location of the other data items.

Following are some commonly used data types in PL/SQL

Data Type	Description
Numeric	Numeric values on which arithmetic operations are performed. It includes sub types such as number, decimal, real, float, etc.
Character	Character values on which character operations such as strings are performed. It includes sub types such as char, varchar, varchar2, nvarchar2, etc.
Date and Time	This data type is for displaying the date and time values. The default date format is DD-MM-YY.
Boolean	These include logical values on which logical operations are performed. The logical values are the Boolean values TRUE and FALSE and the value NULL.

Number	Number(Precision, Scale). Precision is the total number of digits and scale is the number of digits to the right of the decimal point. You cannot use constants or variables to specify precision and scale; you must use integer literals. To declare floating-point numbers, for which you cannot specify precision or scale because the decimal point can float to any position, use the following form without precision and scale : NUMBER The maximum precision that can be specified for a NUMBER value is 38 decimal digits. If you do not specify precision, it defaults to 39 or 40, or the maximum supported by your system, whichever is less. Scale, which can range from -84 to 127, determines where rounding occurs.
Float	ANSI and IBM specific floating-point type with maximum precision of 126 binary digits (approximately 38 decimal digits).
Integer	ANSI and IBM specific integer type with maximum precision of 38 decimal digits.
Real	Floating-point type with maximum precision of 63 binary digits (approximately 18 decimal digits).
Varchar2	Variable-length character string with maximum size of 32,767 bytes.
Rowid	Physical row identifier, the address of a row in an ordinary table.
Blob	This data types is used to store large binary objects in the database. Memory Capacity : 8 to 128 TB.
Clob	This data type is used to store large blocks of character data in the database. Memory Capacity : 8 to 128 TB.

3.5 PL/SQL Variables

- The variables in PL/SQL are declared in declaration section. The syntax of variable declaration is as follows -

Syntax

Variable-Name DataType(Precision/Dimension)

Example

```
Person_name varchar2(30);
```

Here Person_name is a variable name and varchar2 is the data type with 30 as dimension.

- Constraints are associated with the variables defined in the code block. A constraint is a condition that is placed on the variable. Two commonly used constraints are –
 - **Constant** - This constraint will ensure that the value is not changed after a value is initially assigned to a variable. If a statement tries to change the variable value, an error will be displayed.
 - **Not Null** - This constraint will ensure that the variable always contains a value. If the statement attempts to assign an empty or a null value to that particular variable, the program will be error prone and will get abnormal termination of the program or the exception section will execute, if included in the program code.
 - **For Example -**

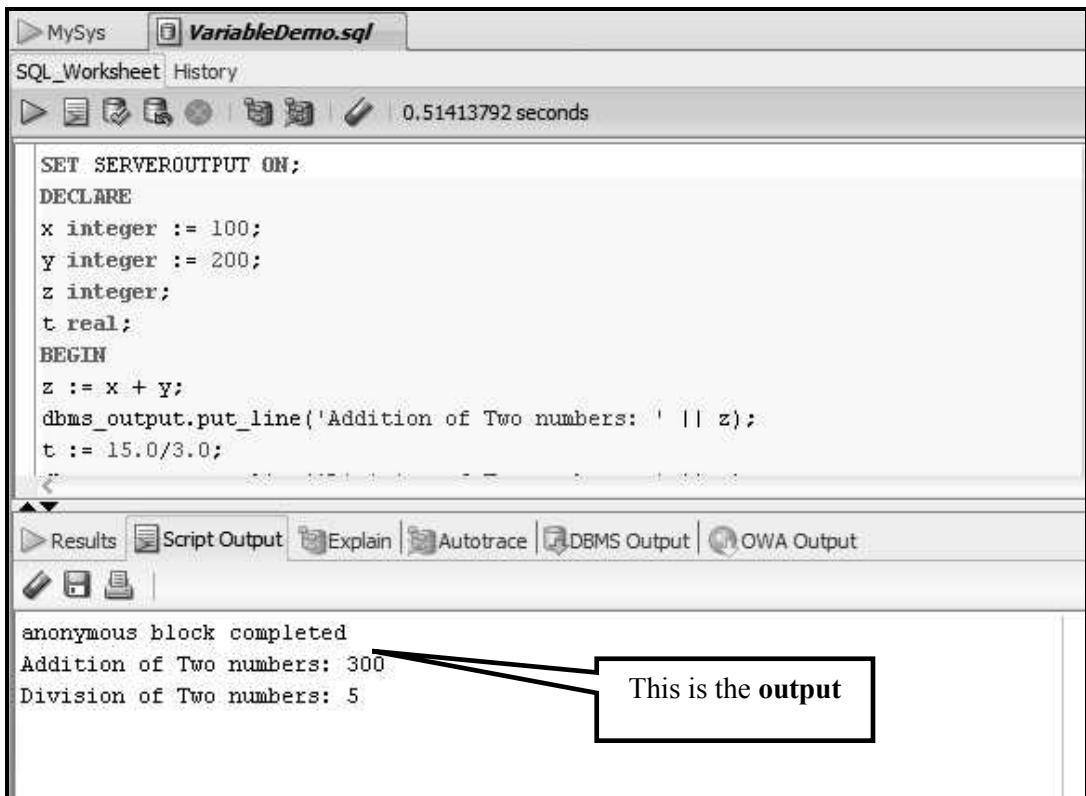
```
PI constant number(5,2)=3.1415
```

- **Variable Initialization** : Variable can be initialized using :=. This sign includes a 'colon' with a succeeding 'equal to' sign. This particular sign assigns the parameter on the right hand side of the sign to the parameter or the variable on the left hand side of the sign.

Programming Example

```
SET SERVEROUTPUT ON;
DECLARE
x integer := 100;
y integer := 200;
z integer;
t real;
BEGIN
z := x + y;
dbms_output.put_line('Addition of Two numbers: ' || z);
t := 15.0/3.0;
dbms_output.put_line('Division of Two numbers: ' || t);
END;
/
```

Following Screenshot demonstrates above program



There are two types of variable scope :

1. Local Variables
2. Global Variables

For example

```
DECLARE
-- Global Variables
a integer := 100;
b integer := 200;
dbms_output.put_line('a: ' || a);
dbms_output.put_line('b: ' || b);
BEGIN
-- Local Variables
c integer := 300;
d integer := 400;
dbms_output.put_line('c: ' || c);
dbms_output.put_line('d: ' || d);
END;
/
```

3.6 PL/SQL Constants

Constant is a value that remains unchanged. It is declared using the keyword **CONSTANT**. It requires initial value that does not get changed.

Syntax

```
constant_name CONSTANT datatype := VALUE;
```

Example

```
PI CONSTANT NUMBER :=3.1415;
```

3.7 Control Statements

The control states determine the flow of execution of PL/SQL block. The control statements are of two types – conditional statements and loop statements.

3.7.1 IF Statement

The If-then-else type of statement is used to specify the condition in PL/SQL block. There are various ways by which the if statement can be written. Their syntaxes are as given below

(1) Syntax- IF-THEN

```
IF condition  
THEN  
Statement: {when condition is true, this statement executes}  
END IF;
```

(2) Syntax- IF-THEN-ELSE

```
IF condition  
THEN  
    {statements to execute when condition is true}  
ELSE  
    {statements to execute when condition is False}  
END IF;
```

(3) Syntax- If-THEN-ELSIF

```
IF condition#1  
THEN  
    {statements to execute when condition#1 is True}  
ELSIF condition#2  
THEN  
    {statements to execute when condition#2 is True}  
END IF;
```

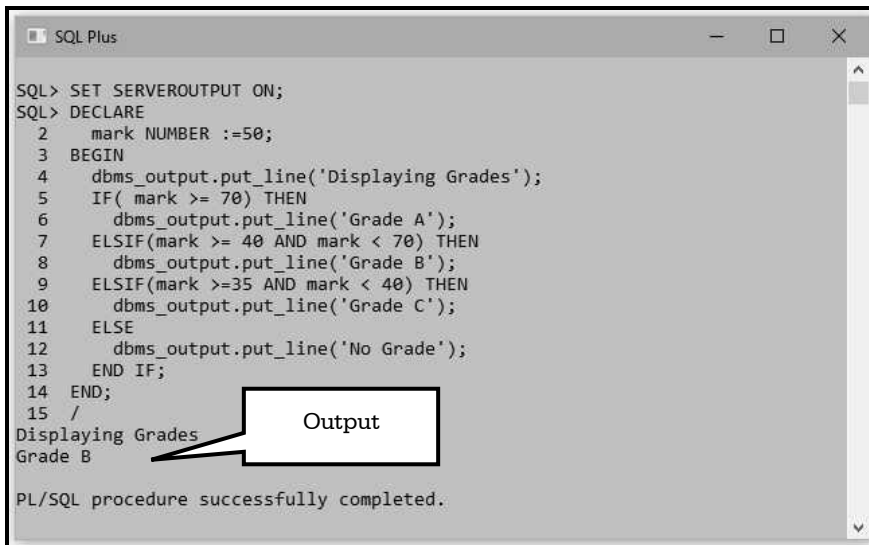
(4) Syntax- If-THEN-ELSIF -ELSE

```
IF condition#1
THEN
    {statements to execute when condition#1 is True}
ELSIF condition#2
THEN
    {statements to execute when condition#2 is True}
ELSE
    {statements to execute when condition#1 and Condition#1 is False}
END IF;
```

Programming Example

```
SET SERVEROUTPUT ON;
DECLARE
    mark NUMBER :=50;
BEGIN
    dbms_output.put_line('Displaying Grades');
    IF( mark >= 70) THEN
        dbms_output.put_line('Grade A');
    ELSIF(mark >= 40 AND mark < 70) THEN
        dbms_output.put_line('Grade B');
    ELSIF(mark >=35 AND mark < 40) THEN
        dbms_output.put_line('Grade C');
    ELSE
        dbms_output.put_line('No Grade');
    END IF;
END;
/
```

Output(Using SQL PLUS)



```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2  mark NUMBER :=50;
  3  BEGIN
  4  dbms_output.put_line('Displaying Grades');
  5  IF( mark >= 70) THEN
  6  dbms_output.put_line('Grade A');
  7  ELSIF(mark >= 40 AND mark < 70) THEN
  8  dbms_output.put_line('Grade B');
  9  ELSIF(mark >=35 AND mark < 40) THEN
 10  dbms_output.put_line('Grade C');
 11  ELSE
 12  dbms_output.put_line('No Grade');
 13  END IF;
 14  END;
 15  /
Displaying Grades
Grade B
PL/SQL procedure successfully completed.
```

Output

Output(Using Oracle SQL Developer)

```
anonymous block completed  
Displaying Grades  
Grade B
```

3.7.2 General Loop

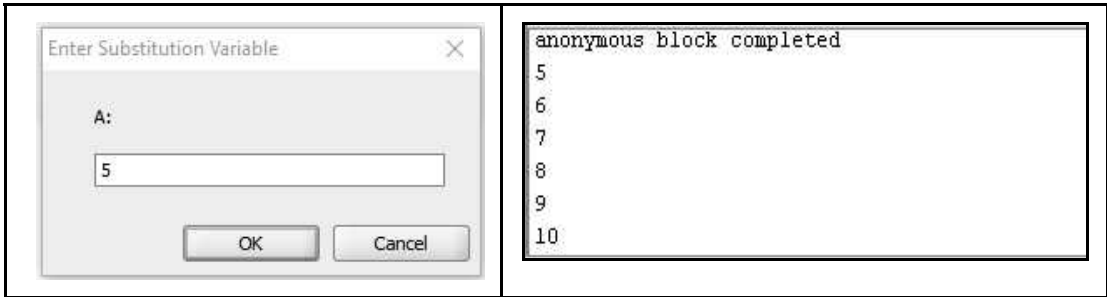
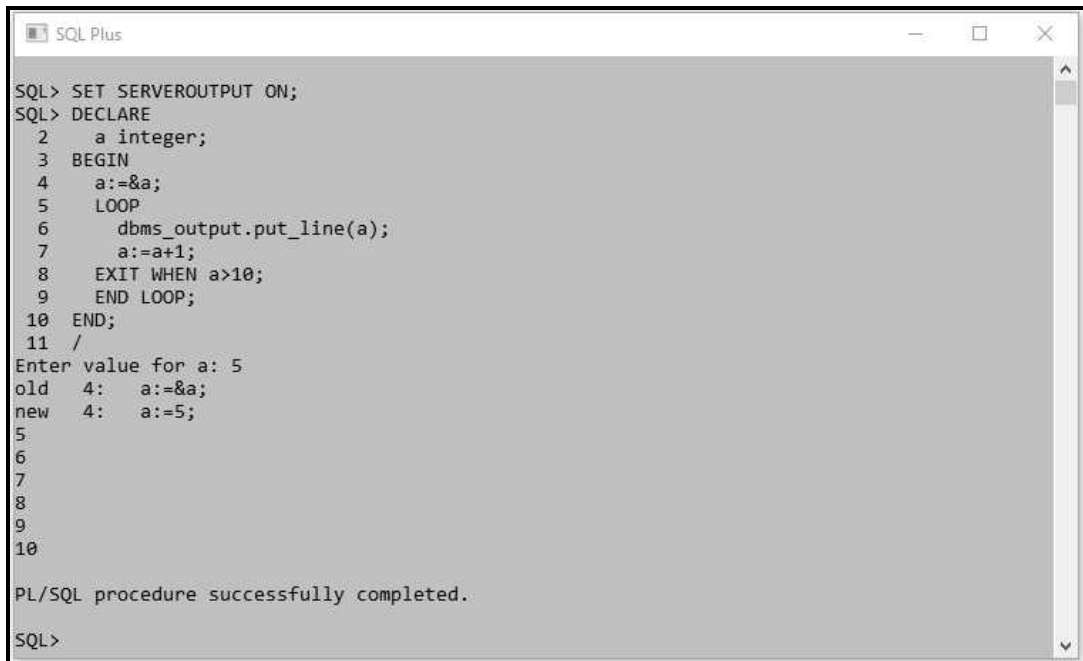
- A General Loop in PL/SQL is used to execute a set of statements at least once before the termination of the loop.
- An EXIT condition has to be specified in the loop; otherwise the looping process will get into a never ending loop, also known as an Infinite Loop
- On encountering with EXIT statement, the loop exits.
- In a PL/SQL Loop, the statements are enclosed between the keywords LOOP and END LOOP.
- In every Loop, the statements are executed and then control restarts from the top of the loop until a certain condition is satisfied.

Syntax

```
LOOP  
  Statement1;  
  Statement2;  
  EXIT;  
END LOOP;
```

Programming Example

```
SET SERVEROUTPUT ON;  
DECLARE  
  a integer;  
BEGIN  
  a:=&a;  
  LOOP  
    dbms_output.put_line(a);  
    a:=a+1;  
    EXIT WHEN a>10;  
  END LOOP;  
END;  
/
```


Output (Using ORACLE SQL DEVELOPER)**Output(Using SQL PLUS)****3.7.3 For Loop**

- For loop executes sequence of statements for specific number of times
- There are starting and ending values in between which the statement executes.
- The counter is incremented by one each time.

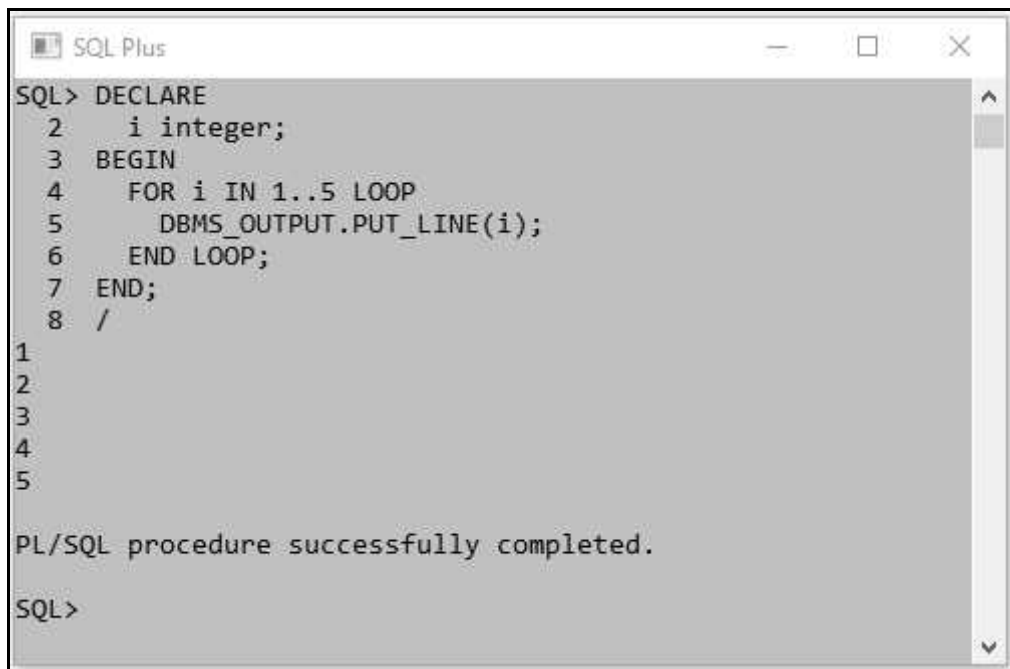
Syntax

```
FOR counter IN num1....num2 LOOP
statement1;
statement2;
END LOOP;
```

Programming Example

```
DECLARE
  i integer;
BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
/
```

Output



```
SQL Plus
SQL> DECLARE
  2   i integer;
  3   BEGIN
  4     FOR i IN 1..5 LOOP
  5       DBMS_OUTPUT.PUT_LINE(i);
  6     END LOOP;
  7   END;
  8   /

1
2
3
4
5

PL/SQL procedure successfully completed.

SQL>
```

3.7.4 While Loop

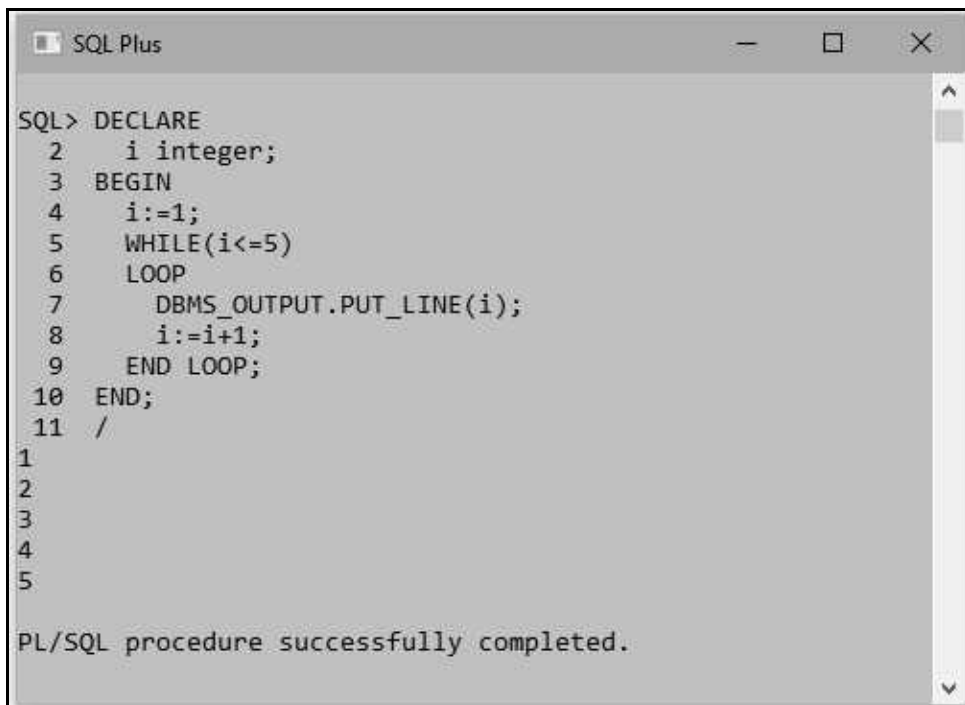
- This is a kind of loop that repeats the sequence of statements while given condition is true.
- Before entering the loop the condition is tested and if it is true then only the control executes the loop body. It executes until the condition becomes false.

Syntax

```
WHILE <condition>
LOOP statement1;
  statement2;
END LOOP;
```

Programming Example

```
DECLARE
  i integer;
BEGIN
  i:=1;
  WHILE(i<=5)
  LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i:=i+1;
  END LOOP;
END;
/
```

Output

```
SQL> DECLARE
  2   i integer;
  3   BEGIN
  4     i:=1;
  5     WHILE(i<=5)
  6     LOOP
  7       DBMS_OUTPUT.PUT_LINE(i);
  8       i:=i+1;
  9     END LOOP;
 10   END;
 11   /

1
2
3
4
5

PL/SQL procedure successfully completed.
```

3.7.5 CASE Statement

- The CASE statement is just similar to switch case statement in C.
- It allows you to execute the sequence of statements based on selector. The selector can be a variable, or a function or some expression.
- A CASE statement is evaluated from top to bottom. If it get the condition TRUE, then the corresponding THEN clause is executed and the execution goes to the END CASE clause.

Syntax

```
CASE [expression]
WHEN condition1 THEN statement1
  WHEN condition2 THEN statement2
  ...
  WHEN conditionn THEN statementn
ELSE someStatement
END
```

Programming Example

```
SET SERVEROUTPUT ON;
DECLARE
  grade CHAR(1) := 'A';
BEGIN
  CASE grade
  WHEN 'A' THEN
    DBMS_OUTPUT.PUT_LINE('Distinction');
  WHEN 'B' THEN
    DBMS_OUTPUT.PUT_LINE('First Class');
  WHEN 'C' THEN
    DBMS_OUTPUT.PUT_LINE('Higher Second Class');
  WHEN 'D' THEN
    DBMS_OUTPUT.PUT_LINE('Second Class');
  WHEN 'F' THEN
    DBMS_OUTPUT.PUT_LINE('Pass');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Fail');
  END CASE;
END;
/
```

Output

```
Distinction
```

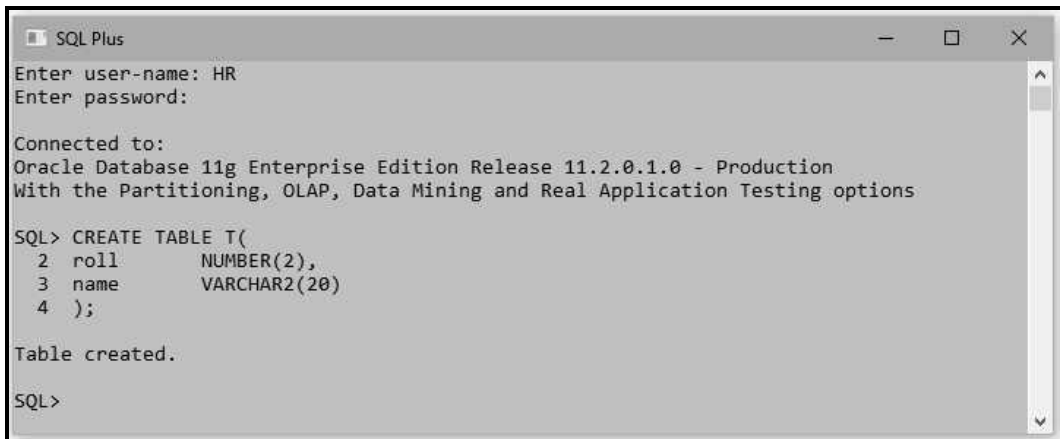
3.8 Handling Database Tables using PL/SQL

We can create a database table in Oracle database using two ways either using SQL prompt or using Oracle SQL Developer.

(1) Creating a Database Table Using SQL PLUS

Open SQL PLUS window, type the user-name and password. I am using HR schema under which the table will be created. The password which I have set during installation of ORACLE is HR. So by entering correct user name and password the SQL prompt will appear. Just give the CREATE table query for creating the table.

Following screenshot illustrates creation of table named T with two fields - **roll** and **name**.



```
SQL Plus
Enter user-name: HR
Enter password:

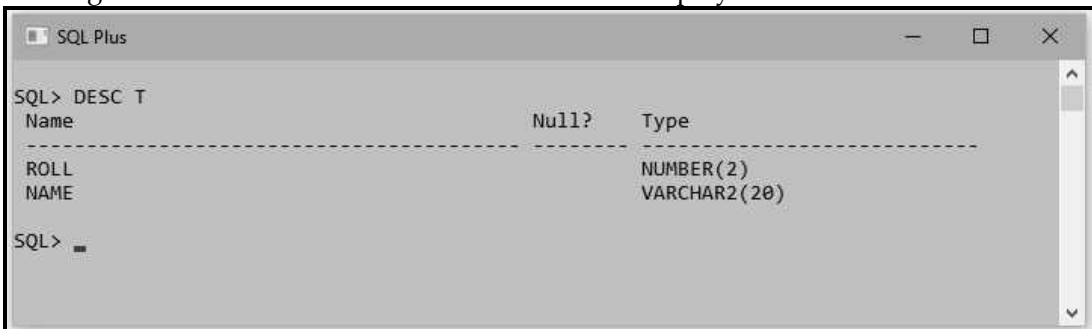
Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> CREATE TABLE T(
2  roll      NUMBER(2),
3  name      VARCHAR2(20)
4  );

Table created.

SQL>
```

Using DESC command the table structure can be displayed.

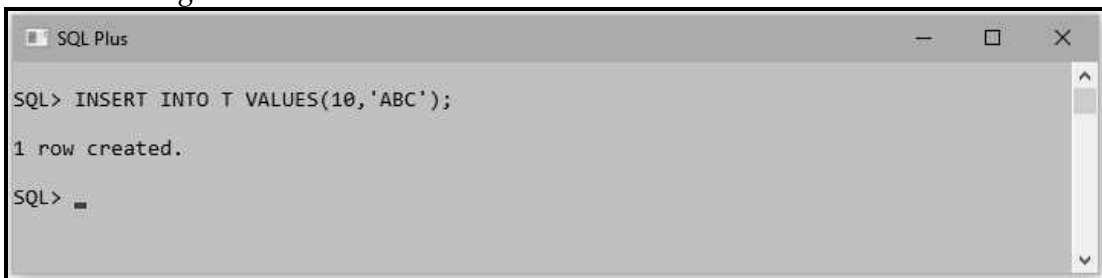


```
SQL Plus

SQL> DESC T
Name                                Null?    Type
-----
ROLL                                NUMBER(2)
NAME                                VARCHAR2(20)

SQL>
```

For inserting the values into the table we use INSERT command as follows -



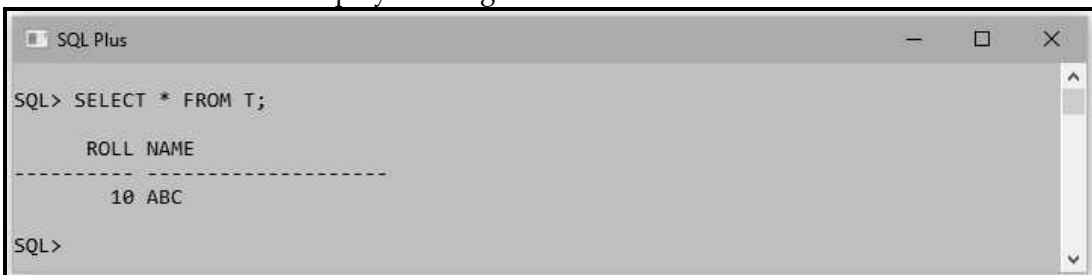
```
SQL Plus

SQL> INSERT INTO T VALUES(10, 'ABC');

1 row created.

SQL>
```

The table can then be displayed using SELECT command as follows -



```
SQL Plus

SQL> SELECT * FROM T;

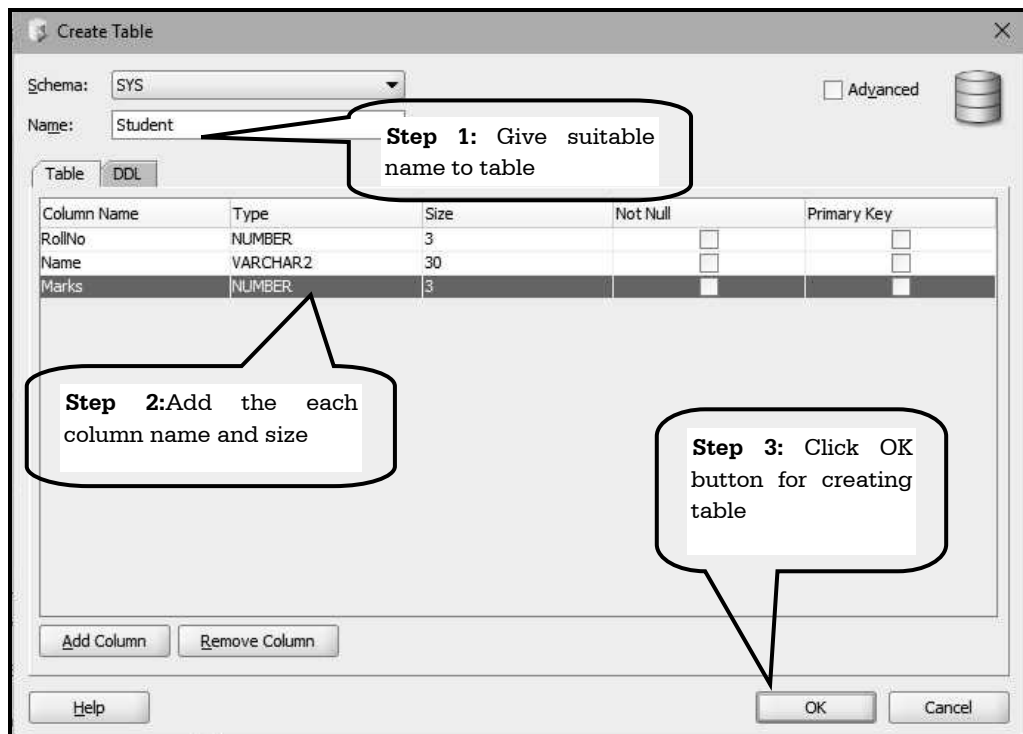
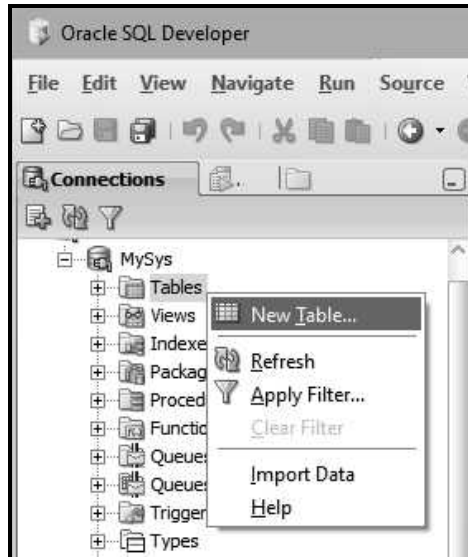
  ROLL NAME
-----
   10 ABC

SQL>
```

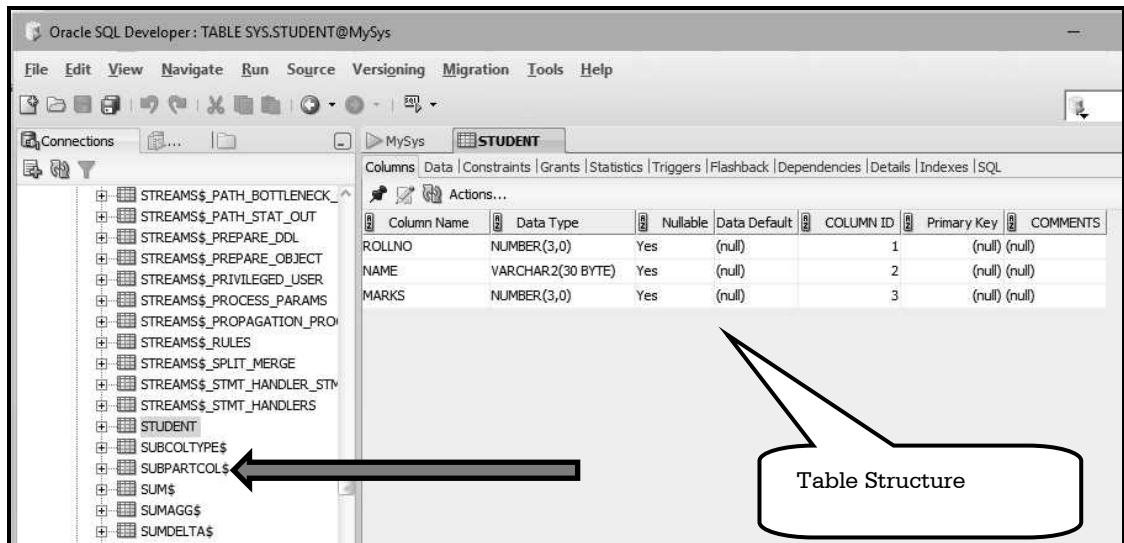
(2) Creating a Database Table Using SQL Developer

Open Oracle SQL Developer and follow these steps -

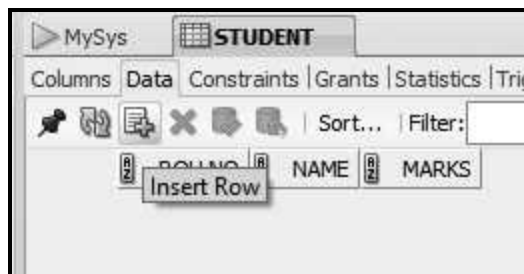
Step 1 : Expand your Database Connection node. Find out the node Table. Just Right click on it, click on New Table. Here I have already created a connection by name MySys (Refer section A.3(2) to know how to create a connection), so I am creating a table under this schema. Here is the screenshot.



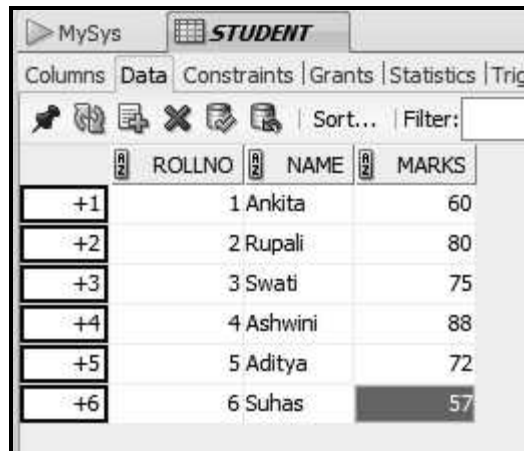
Now you can see your table structure by Selecting Student Table from Table node of your database Connection List. Just refer following screenshot.



Just click on **Data** tab for insertion of data into the table -



The data can be inserted into the table by clicking to enter the value to each column -



Then click Save button. The data will be inserted into the table.

3.9 Examples on PL/SQL

Example 3.9.1 Write PL/SQL block of code which accepts the roll.no. from user, the attendance of roll no entered by user will be checked in stud_att (Roll_no, Att) table. Attendance of Roll no entered is displayed on screen.

SPPU : Aug.-17, End Sem, Marks 5

Solution :

Create Table

```
CREATE TABLE TESTUDENTS(Roll_no NUMBER(3)PRIMARY KEY, att NUMBER(5));
```

Insert Data

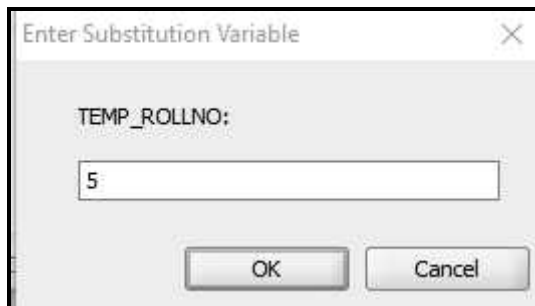
```
INSERT INTO "SYS"."TESTUDENTS" (ROLL_NO, ATT) VALUES ('1', '130')
INSERT INTO "SYS"."TESTUDENTS" (ROLL_NO, ATT) VALUES ('2', '155')
INSERT INTO "SYS"."TESTUDENTS" (ROLL_NO, ATT) VALUES ('3', '180')
INSERT INTO "SYS"."TESTUDENTS" (ROLL_NO, ATT) VALUES ('4', '100')
INSERT INTO "SYS"."TESTUDENTS" (ROLL_NO, ATT) VALUES ('5', '98')
INSERT INTO "SYS"."TESTUDENTS" (ROLL_NO, ATT) VALUES ('6', '190')
```

The table can be viewed as follows –

R	ROLL_NO	ATT
1	1	130
2	2	155
3	3	180
4	4	100
5	5	98
6	6	190

PL/SQL Program(Without Function)

```
SET SERVEROUTPUT ON;
DECLARE
temp_rollno NUMBER(3);
temp_att NUMBER(5);
total_days NUMBER(5) := 200;
BEGIN
temp_rollno := &temp_rollno;
SELECT att INTO temp_att FROM TESTUDENTS WHERE roll_no = temp_rollno;
DBMS_OUTPUT.put_line('Roll No = ' || temp_rollno || ' Attendance = ' || temp_att);
END;
/
```


Output

```
anonymous block completed  
Roll No = 5 Attendance = 98
```

Example 3.9.2 Write a PL/SQL code block for displaying the area of circle with radius values from 3 to 8. (Formula to calculate area of circle is $\text{Pi} \times \text{radius} \times \text{radius}$).

SPPU : Oct.-18, In Sem, Marks 5

Solution :

```
SET SERVEROUTPUT ON;  
DECLARE  
  -- The radius is declared in 'radius' variable and result in 'area' variable  
  radius NUMBER;  
  area NUMBER(10,2);  
BEGIN  
  DBMS_OUTPUT.put_line('Radius   | Area of circle ');  
  DBMS_OUTPUT.put_line('-----');  
  FOR radius IN 3..8 LOOP  
    area:=3.14* radius* radius;  
    DBMS_OUTPUT.put_line('    || radius || ' | ' || area);  
  END LOOP;  
END;  
/
```

Output

ResultsScript OutputExplainAutotraceDBMS OutputOWA Output

anonymous block completed

Radius	Area of circle
3	28.26
4	50.24
5	78.5
6	113.04
7	153.86
8	200.96

Example 3.9.3 Write the PL/SQL block of code to calculate the factorial value of a number.

SPPU : May-19, Dec.-19, End Sem, Marks 5

Solution :

```
SET SERVEROUTPUT ON;
DECLARE
-- The factorial f is initialized to 1
f NUMBER := 1;
--user inputs the number here in variable n
n NUMBER := &1;
num NUMBER := n;
BEGIN
WHILE n>0 LOOP
f:=n*f;
n:=n-1;
END LOOP;
DBMS_OUTPUT.put_line('Factorial of ' || num || ' is ' || f);
END;
/
```

Output

Factorial of 7 is 5040

Example 3.9.4 Write PI/SQL block of code for following requirement :

Student_fees (PRN, S_name, class, fees_paid).

Accept the PRN of student from user, check the fees paid by student, if fees paid is less than 30,000 then display the message on screen not paid full fees and display the total fees due. If fees_paid is greater than or equal to 30,000 then display message no fees due.

SPPU : Oct.-19, In Sem, Marks 5

Solution : Step 1 : Create Table using following SQL command

```
CREATE TABLE STUDENT_FEES
(PRN NUMBER(5),
```

```

S_NAME VARCHAR2(30),
CLASS VARCHAR2(20),
FEES_PAID NUMBER(10)
};

```

Insert some data into this table. The sample table can be viewed as

PRN	S_NAME	CLASS	FEES_PAID
101	AAA	TEComp	30000
102	BBB	SEMech	25000
103	CCC	BECivil	35000
104	DDD	FE	28000

Step 2 : Following is the PL/SQL code that checks whether the fees paid fully or not.

PL/SQL Program

```

SET SERVEROUTPUT ON;
DECLARE
temp_PRN NUMBER(5);
temp_fees NUMBER(10);
BEGIN
temp_PRN := &temp_PRN;
SELECT fees_paid INTO temp_fees FROM STUDENT_FEES WHERE PRN = temp_PRN;
DBMS_OUTPUT.put_line('PRN = ' || temp_prn);
DBMS_OUTPUT.put_line('Fees(Rs.) = ' || temp_fees);
IF(temp_fees<30000) THEN
DBMS_OUTPUT.put_line('The Feees not Paid Fully');
ELSE
DBMS_OUTPUT.put_line('No Fees Due');
END IF;
END;
/

```

Output(Run1)

```

PRN = 101
Fees(Rs.) = 30000
No Fees Due

```

3.10 Concept of Stored Procedures

- Stored procedure is a type of subprogram in PL/SQL block. It is a group of statements that can be called by its name.
- This is a subprogram that does not return a value directly.
- A procedure is created with the CREATE OR REPLACE PROCEDURE statement.

Syntax

```
CREATE or REPLACE Procedure Procedure_Name  
[(Parameter_Name [IN | OUT | IN OUT ] Type [...])]  
[IS | AS]  
BEGIN  
Procedure_Body  
END;
```

- Where
 - **Procedure_Name** is used to specify the name of the procedure.
 - **CREATE** keyword is used to develop a new procedure and [OR REPLACE] option allows us to modify an existing procedure.
 - The optional **parameter list** contains name and types of parameters.
- There are three **ways to pass parameters in procedures** -
 - **IN** represents that argument value will be passed from outside the procedure. It is a read-only parameter. Parameters are passed by reference. Inside the procedure or a sub-program, an IN Parameter acts as a constant. It cannot be assigned a value. It is the default mode of parameter passing.
 - An **OUT** parameter returns a value to the calling program. OUT represents that this parameter will be used to return a value outside of the procedure.
 - An **IN OUT** parameter passes an initial value to the sub-program and returns an updated value to the caller. We can read and write values using this parameter.
- The executable part is included in the procedure body.
- We can create a procedure without passing the parameters or by passing the parameters.
- We can execute the procedure using **Execute** keyword.
- A standalone procedure can be called by using the EXECUTE keyword or by calling or mentioning the procedure by its name from a PL/SQL block.

Syntax

```
Execute [Procedure-Name];
```

3.10.1 Procedures without Parameter

Let us write a simple procedure for displaying some welcome message.

Step 1 : Create a simple sql file that contains a procedure

```
SET SERVEROUTPUT ON;  
CREATE OR REPLACE PROCEDURE MyMessage ← Creation of procedure
```

```
IS
BEGIN
    dbms_output.put_line('Welcome friend!!!');
END;
/
EXECUTE MyMessage; ← This is for executing the procedure
```

Step 2 : Save and compile the code. The output will be as follows -

```
PROCEDURE MyMessage Compiled.
anonymous block completed
Welcome friend!!!
```

Another way of execution of procedure

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE MyMessage ← Creation of procedure
IS
BEGIN
    dbms_output.put_line('Welcome friend!!!');
END;
/
BEGIN ← This is for calling the procedure
    MyMessage;
END;
/
```

Save and compile above code to get the output.

3.10.2 Procedures with Parameters

Following example show the procedure for addition of two numbers. The result is stored in third variable which is an output variable.

Step 1 : We will create a procedure and store it in an sql file as follows

```
CREATE OR REPLACE PROCEDURE AddProc(x IN NUMBER, y IN NUMBER, z OUT NUMBER)
IS
BEGIN
    z:=x+y;
END;
/
```

Two Input and One output
parameter

Save and compile above code.

Step 2 : The call to the procedure can be made from another file. The code for calling the procedure for execution is as follows -

```
SET SERVEROUTPUT ON;
DECLARE
a NUMBER;
b NUMBER;
c NUMBER;

BEGIN
  a := 10;
  b := 20;
  AddProc(a,b,c);
  dbms_output.put_line('Addition of two numbers:' || c);
END;
```

Save and compile above code

Step 3 : The output will be as follows -

```
anonymous block completed
Addition of two numbers:30
```

3.10.3 Stored Procedure for Handling Database Table

Using a stored procedure we can access the database table and can perform insertion, deletion and updation of data.

Following is a simple example of procedure using which we insert some record in student table.

Step 1 : Write a procedure as follows

```
CREATE OR REPLACE PROCEDURE InsertStudent(roll IN NUMBER,
name IN VARCHAR2,
marks IN NUMBER,
branch IN VARCHAR2)
IS
BEGIN
INSERT INTO student VALUES(roll,name,marks,branch);
END;
/
```

Save and compile the above code.

Step 2 : Now call the procedure. The code is as follows

```
SET SERVEROUTPUT ON;
BEGIN
  InsertStudent(7,'Akash',78,'CIVIL');
  DBMS_OUTPUT.PUT_LINE('One row inserted in Student Table!!!');
END;
/
```

Save and compile the above code.

Step 3 : The output can be as follows

```
anonymous block completed
One row inserted in Student Table!!!
```

Step 4 : You can cross-verify the above execution by opening the student table. It should display new row inserted into it.

R	ROLLNO	R	NAME	R	MARKS	R	BRANCH
1	1	Ankita		60	CS		
2	2	Rupali		80	Mechanical		
3	3	Swati		75	Electrical		
4	4	Ashwini		88	Civil		
5	5	Aditya		72	IT		
6	6	Suhas		57	Electronics		
7	7	Akash		78	CIVIL		

Inserted new row

3.11 Functions

SPPU : Aug.-17, Marks 5

- Stored function is a named block or subprogram in PL/SQL.
- In PL/SQL, a function takes one or more parameter and returns one value.
- The syntax for a function in PL/SQL is as follows :

```
CREATE [Or REPLACE] Function Function_Name
[(Parameter,...)]
Return Datatype
[IS | AS]
[Declaration Section]
BEGIN
```

```
[Executable Section]
END Function_Name;
```

Let us now understand how to write a function in PL/SQL with the help of example

Step 1 : Open New file in Oracle SQL Developer and write following code for creating a simple function for addition of two numbers.

AdditionFunction.sql

```
CREATE OR REPLACE FUNCTION Addion_fun(a IN NUMBER, b IN NUMBER)
RETURN NUMBER IS
c NUMBER;
BEGIN
  c := a+b;
  RETURN c;
END;
/
```

Save this file and compile it.



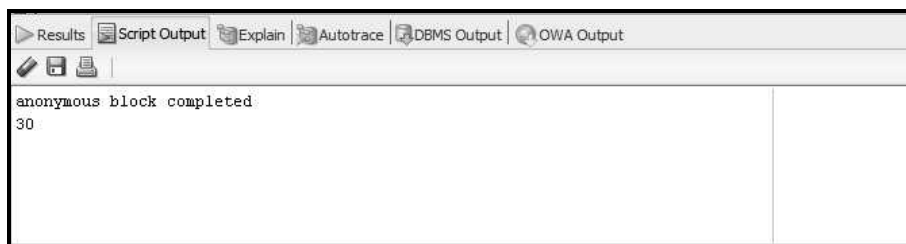
Step 3 : Create another sql file for calling the above function in an anonymous block.

CallAdditionFun.sql

```
SET SERVEROUTPUT ON;
BEGIN
  DBMS_OUTPUT.PUT_LINE(addion_Fun(10,20));
END;
/
```

Save above code and compile it.

Step 4 : On execution of the above code you get the output as



3.11.1 PL/SQL Stored Function for Table

We can write PL/SQL stored Function can be used to perform some functionality on the database tables. Following example shows that we can write a function to find total number of students from **STUDENT** table. Here is a stepwise illustration -

Step 1 : Create a student table as follows -

ROLLNO	NAME	MARKS	BRANCH
1	Ankita	60	CS
2	Rupali	80	Mechanical
3	Swati	75	Electrical
4	Ashwini	88	Civil
5	Aditya	72	IT
6	Suhas	57	Electronics

Step 2 : Now create an sql file and create a function in it.

DisplayTotalFun.sql

```
CREATE OR REPLACE FUNCTION Total_Students
RETURN NUMBER IS
  t NUMBER(2) := 0;
BEGIN
  SELECT COUNT(*) INTO t
  FROM student;
  RETURN t;
END;
/
```

Save and compile above program

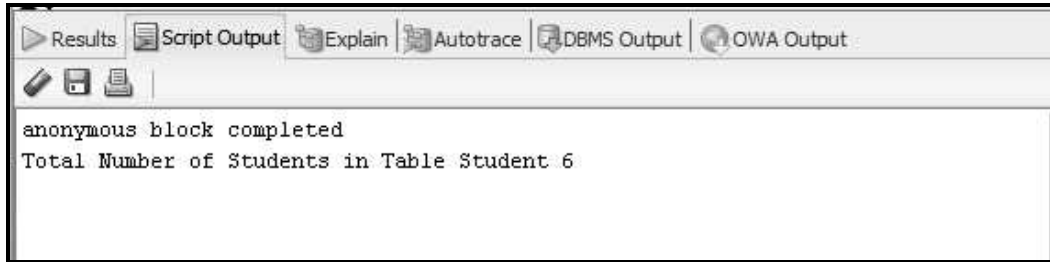
```
FUNCTION Total_Students Compiled.
```

Step 3 : Now we will write sql file for calling the above function.

CallFun.sql

```
SET SERVEROUTPUT ON;
DECLARE
  t NUMBER(2);
BEGIN
  t := Total_Students();
  DBMS_OUTPUT.PUT_LINE('Total Number of Students in Table Student ' || t);
END;
/
```

Save above code and compile it.



Example 3.11.1 Write a PL/SQL function that returns the name of the student whose roll number is given by user.

Solution :

Step 1 : Create a student table. The sample table is as follows -

ROLLNO	NAME	MARKS	BRANCH
1	Ankita	60	CS
2	Rupali	80	Mechanical
3	Swati	75	Electrical
4	Ashwini	88	Civil
5	Aditya	72	IT
6	Suhas	57	Electronics

Step 2 : Write a function for retrieving the name of the student from the STUDENT table

```
CREATE OR REPLACE FUNCTION Display(roll NUMBER)
RETURN VARCHAR2 IS
    s_name VARCHAR2(20);
BEGIN
    SELECT name INTO s_name
    FROM student
    WHERE ROLLNO=roll;
    RETURN s_name;
END;
/
```

Save and compile above code.

Step 3 : Now call the above function

```
SET SERVEROUTPUT ON;
DECLARE
    s_name VARCHAR2(20);
BEGIN
    s_name := Display(2);
    DBMS_OUTPUT.PUT_LINE('Name Student with Rollno 2 is: ' || s_name);
END;
/
```

Save and compile above file. You will get following output



Example 3.11.2 Write PL/SQL block of code which accepts the roll.no. from user, the attendance of roll no entered by user will be checked in stud_att (Roll_no, Att) table. Attendance of Roll no entered is displayed on screen.

SPPU : Aug.-17, End Sem, Marks 5

Solution : PL/SQL Program

Step 1 : Create a function in a sql file as

example.sql

```
CREATE OR REPLACE FUNCTION Display(roll NUMBER)
RETURN NUMBER IS
    temp_att NUMBER(5);
BEGIN
    SELECT att INTO temp_att FROM TESTUDENTS WHERE roll_no = roll;
    RETURN temp_att;
END;
/
```

Step 2 : Following is a driver program that calls above created function

driver.sql

```
SET SERVEROUTPUT ON;
DECLARE
    temp_rollno NUMBER(3);
    temp_att NUMBER(5);
    total_days NUMBER(5) := 200;
BEGIN
    temp_rollno := &temp_rollno;
    temp_att := Display(temp_rollno);
    DBMS_OUTPUT.put_line('Roll No = ' || temp_rollno || ' Attendance = ' || temp_att);
END;
/
```

Output

Roll No = 4 Attendance = 100

3.12 Cursors

SPPU : Dec.-17, Marks 5

- When an SQL statement is processed, Oracle creates a memory area known as **context area**. A cursor is a pointer to this context area.

- It contains all information needed for processing the statement.
- In PL/SQL, the context area is controlled by cursor.
- A cursor contains information on a SELECT statement and the rows of data accessed by it.
- The cursor is used to fetch and process the rows returned by SQL statement one at a time.
- There are two types of cursors -
(1) Implicit cursor (2) Explicit cursor

(1) Implicit cursor

- Whenever Oracle executes an SQL statement such as SELECT INTO, INSERT, UPDATE and DELETE, it creates an implicit cursor.
- The Implicit Cursor is the Default Cursor in PL/SQL block.
- Oracle provides some attributes known as Implicit cursor's attributes to check the status of DML operations. Some of them are as follows -
 - **%ROWCOUNT** : It returns the number of rows influenced by an UPDATE, DELETE or an INSERT statement.
 - **%NOTFOUND** : It returns TRUE if an UPDATE, DELETE or an INSERT statement influenced zero rows or a SELECT INTO statement returned no rows. Else, it returns a FALSE.
 - **%FOUND** : It returns TRUE if an UPDATE, DELETE or an INSERT statement influenced one or more rows or a SELECT INTO statement returned one or more rows. Else, it returns FALSE.
 - **%ISOPEN** : It returns FALSE for Implicit cursors as Oracle closes the PL/SQL cursor by default after processing its associated PL/SQL statement.

Programming Example : Following is an example program that uses attributes of implicit cursor to demonstrate number of rows affected when SELECT query is executed for Student table

```
SET SERVEROUTPUT ON;
DECLARE
  s_name student.name%type;
BEGIN
  SELECT name INTO s_name
  FROM student
  WHERE marks >= 70;
  DBMS_OUTPUT.PUT_LINE('Number of rows processed: ' || sql%rowcount);
END;
/
```

Output

```
Number of rows processed:5
```

(2) Explicit cursor

- Explicit cursors are used when you are executing a SELECT statement query that will return more than one row.
- Cursors can process one record at a given point of time even though it stores more than one record.
- An explicit cursor is defined in the declaration section of the PL/SQL Block.
- Explicit cursors are used if you need to have better control over the Context Area via Cursor.

Syntax for Declaration of Explicit Cursor

```
CURSOR cursor_name  
IS  
SELECT statement
```

For example

```
CURSOR MyCursor  
IS  
SELECT * FROM Student;
```

The explicit cursor works in four stages -

- 1) **Declaration of cursor** : The declaration of cursor is in declaration section of PL/SQL block. The name of the cursor requires to be defined along with the SELECT Statement.

Syntax

```
CURSOR cursor-name IS  
SELECT statement;
```

- 2) **Open the cursor** : Opening the cursor also means allocating the memory for the cursor in the context area which thereby makes it sufficient to fetch and store records in it.

Syntax

```
OPEN cursor-name;
```

- 3) **Fetch the cursor** : Fetching the cursor involves retrieval of data using the fetch statement. It is used to help the cursor process and access records or rows at a time.

Syntax

```
FETCH cursor_name INTO variable_list;
```

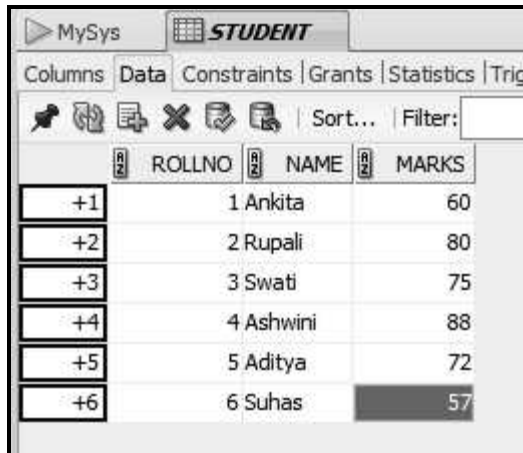
- 4) **Close the cursor** : Once the work associated for a cursor to be completed is accomplished, it is necessary to release the allocated memory of the cursor to let other tasks occupy memory.

Syntax

```
Close cursor_name;
```

Now let us understand how to write a simple explicit cursor and execute it.

Step 1 : Create a database table for using cursor. I have already created a student table in section 3.8 (2). It is as follows :



The screenshot shows the MySys database tool interface. The 'STUDENT' table is selected, and the 'Data' tab is active. The table has three columns: ROLLNO, NAME, and MARKS. The data is as follows:

ROLLNO	NAME	MARKS
1	Ankita	60
2	Rupali	80
3	Swati	75
4	Ashwini	88
5	Aditya	72
6	Suhas	57

Step 2 : Under your current connection type(In my case it is **MySys**), Open **File->New File**. Give suitable name to your file which stores the cursor program. I have given the name as **CursorDemo.sql**

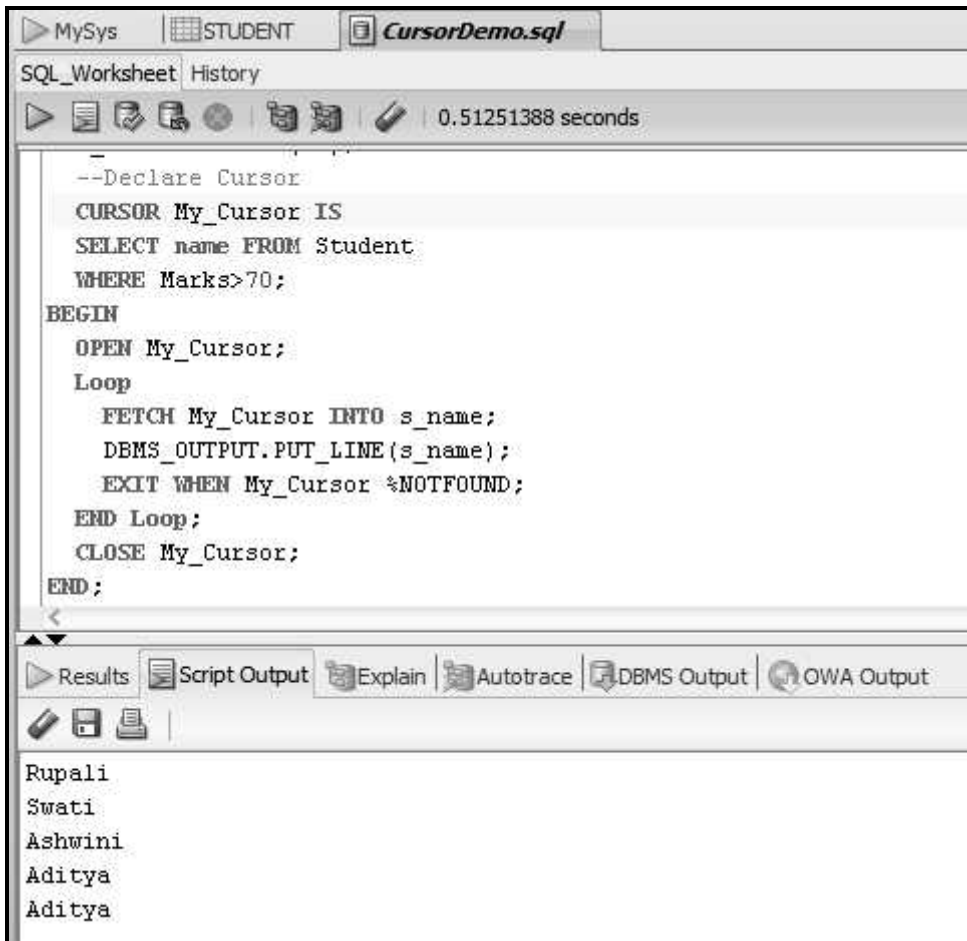
Step 3 : Write the program as follows

```
SET SERVEROUTPUT ON;
DECLARE
  s_name VARCHAR2(30);
  --Declare Cursor
  CURSOR My_Cursor IS
    SELECT name FROM Student
    WHERE Marks>70;
BEGIN
  OPEN My_Cursor;
  Loop
    FETCH My_Cursor INTO s_name;
    DBMS_OUTPUT.PUT_LINE(s_name);
    EXIT WHEN My_Cursor %NOTFOUND;
  END Loop;
  CLOSE My_Cursor;
END;
```

Step 4 : Save your file and execute script by either hitting F5 or by Run Script



The output will be obtained as a list of names of those students who have marks > 70. Here is the output



```
--Declare Cursor
CURSOR My_Cursor IS
SELECT name FROM Student
WHERE Marks>70;
BEGIN
OPEN My_Cursor;
Loop
  FETCH My_Cursor INTO s_name;
  DBMS_OUTPUT.PUT_LINE(s_name);
  EXIT WHEN My_Cursor %NOTFOUND;
END Loop;
CLOSE My_Cursor;
END;
```

Results | Script Output | Explain | Autotrace | DBMS Output | OWA Output

Rupali
Swati
Ashwini
Aditya
Aditya

Program Explanation : In above code,

- The cursor is created to display the names of students who have scored more than 70 marks.
- The name of the cursor is My_Cursor. The SELECT query associated with it is
`SELECT name FROM STUDENT WHERE Marks > 70;`
- Then we open the My_Cursor.
- Inside the LOOP, we fetch students' name from student table into variable s_name and each time the s_name is displayed. This loop is exited when no record is found. For that purpose the attribute %NOTFOUND is used.
- Finally the cursor is closed.

Example 3.12.1 The organization has decided to increase the salary of employees by 10 % of existing salary, whose existing salary is less than ₹ 10000/-. Write a PL/SQ block to update the salary as per above requirement, display an appropriate message based on the number of rows affected by this update (using implicit cursor status variables).

SPPU : Dec.-17, End Sem, Marks 5

Solution :

Step 1 : Create an employee table as follows -

R#	EMP_NO	R#	EMP_NAME	R#	SALARY
1	101	AAA			25000
2	102	BBB			12000
3	103	CCC			7000
4	104	DDD			5000
5	105	EEE			22000
6	106	FFF			9000

PL/SQL Program

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
UPDATE emp_table
```

```
SET salary = salary+(salary*0.1)
```

```
WHERE salary < 10000;
```

SQL query to increase the salary(which is less than 10000) by 10%.

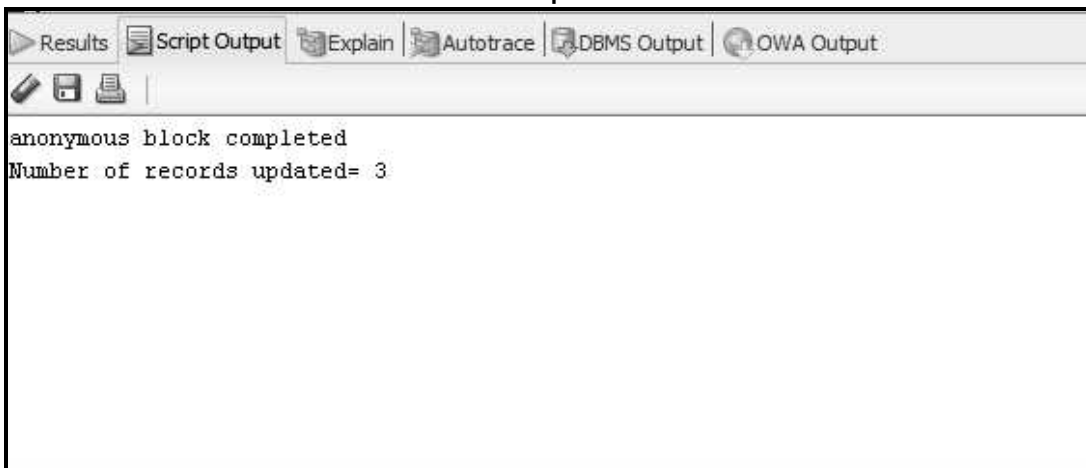
```
DBMS_OUTPUT.put_line('Number of records updated= ' || sql%rowcount);
```

```
END;
```

```
/
```

Use of implicit cursor variable

Output



Now If we open the database table we get the updated values of the salary as

	EMP_NO	EMP_NAME	SALARY
1	101	AAA	25000
2	102	BBB	12000
3	103	CCC	7700
4	104	DDD	5500
5	105	EEE	22000
6	106	FFF	9900



3.13 Triggers

SPPU : Dec.-17, 18, 19, Marks 5

- Trigger is something that is invoked automatically when some event occurs.
- PL/SQL triggers are **block structures or pre-defined programs**, which may be in-built or even explicitly developed by the programmers for a particular task.
- Trigger is **stored into database** and invoked repeatedly, when specific condition match.
- Triggers are stored programs, which are automatically executed or fired when some event occurs.
- Triggers are associated with response-based events such as a,
 - Database Definition Language (DDL) statement such as CREATE, DROP or ALTER .
 - Database Manipulation Language (DML) statement such as UPDATE, INSERT or DELETE.
 - Any other database operation such as a Startup, Shutdown, Logging in and Logging Out.

Syntax

```
CREATE OR REPLACE TRIGGER Trigger_Name
BEFORE or AFTER or INSTEAD OF
INSERT or UPDATE or DELETE
of Column_Name
ON Table_Name
[REFERENCING OLD AS O New AS N]
FOR EACH ROW
WHEN (Condition)
DECLARE
```

```
Declaration Section
BEGIN
Execution Section
END;
```

Where

- **CREATE [OR REPLACE] TRIGGER trigger_name:** It creates or replaces an existing trigger with some trigger_name.
- **{BEFORE | AFTER | INSTEAD OF} :** This specifies when the trigger would be executed. The INSTEAD OF clause is used for creating trigger on a view.
- **{INSERT [OR] | UPDATE [OR] | DELETE} :** This specifies the DML operation.
- **[OF col_name] :** This specifies the column name that would be updated.
- **[ON table_name] :** This specifies the name of the table associated with the trigger.
- **[REFERENCING OLD AS o NEW AS n]:** This allows you to refer new and old values for various DML statements, like INSERT, UPDATE, and DELETE.
- **[FOR EACH ROW] :** This specifies a row level trigger, i.e., the trigger would be executed for each row being affected. Otherwise the trigger will execute just once when the SQL statement is executed, which is called a table level trigger.
- **WHEN (condition) :** This provides a condition for rows for which the trigger would fire. This clause is valid only for row level triggers.

Programming Example

Let us now see how to use trigger with the help of some example -

Example 3.13.1 Write a trigger for insertion of a row into a person table. On firing the trigger name of the user who performed insertion operation should be displayed.

Solution :

```
CREATE TABLE PersonTab (
  pname VARCHAR2(20)
);
```

Here table named **PersonTab** is created

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE TRIGGER MyTrigger
BEFORE INSERT ON PersonTab
FOR EACH ROW
ENABLE
DECLARE
  usr_name VARCHAR2(20);
BEGIN
  SELECT user INTO usr_name FROM dual;
```

Code for actual trigger

```

DBMS_OUTPUT.PUT_LINE('Inserted a new row by user: ' || usr_name);
END;
/

INSERT INTO PersonTab VALUES('Sharda');

```

SQL query on execution of which the trigger gets fired.

Output

```

*Cause:
*Action:
TRIGGER MyTrigger Compiled.
1 rows inserted
Inserted a new row by user: HR

```

You can cross-verify the insertion of data into the table by opening the corresponding table.

For instance -

PNAME
1 Sharda

Example 3.13.2 Write PL/SQL trigger for following requirement :

Event : Deletion of row from stud (roll_no, name, class) table.

Action : After deletion of values from stud table, values should be inserted into cancel_admission (roll_no, name) table

Note : For every row to be deleted, action should be performed.

SPPU : Dec.-17, End Sem, Marks 5

Solution :

Step 1 : Create table **stud_table** using following SQL statement

```

CREATE TABLE stud_table(
    roll_no NUMBER(5),
    sname VARCHAR2(30),
    sclass VARCHAR2(20)
);

```

Step 2 : Insert data into table using following commands

```

INSERT INTO stud_table VALUES(111,'AAA','TECOMP');
INSERT INTO stud_table VALUES(222,'BBB','SEMECH');
INSERT INTO stud_table VALUES(333,'CCC','BECIVIL');
INSERT INTO stud_table VALUES(444,'DDD','TEETC');

```

Step 3 : Create table cancel_admission using following SQL statement

```

CREATE TABLE cancel_admission(
    roll_no NUMBER(5),
    sname VARCHAR2(30)
);

```

Step 4 : Following is a PL/SQL program that deletes data from **stud_table** and insert the deleted data into **cancel_admission** table

TriggerDemo1.sql

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE TRIGGER Stud_AD
AFTER DELETE ON stud_table
FOR EACH ROW
ENABLE

BEGIN
  INSERT INTO cancel_admission
  (roll_no,
   sname)
  VALUES
  ( :old.roll_no,
   :old.sname);
END;
/
DELETE FROM stud_table WHERE roll_no=222
```

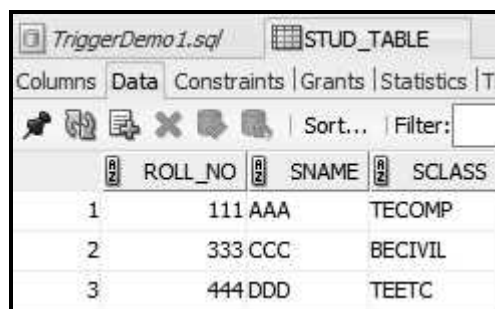
Output

On compiling above trigger we get the output as follows -

```
TRIGGER Stud_AD Compiled.
1 rows deleted
```

The execution of this trigger can be verified by observing the data from student_table and cancel_admission table.

Stud_table Table



	ROLL_NO	SNAME	SCLASS
1	111	AAA	TECOMP
2	333	CCC	BECIVIL
3	444	DDD	TEETC

Cancel_admission Table

ROLL_NO	SNAME
1	222 BBB

Example 3.13.3 Consider the schema : *student_fees_detail* (*name*, *total_fees_deposited*, *till_date*)

Answer the following :

- i) Write a SQL query to display the total fees deposited by students whose minimum 3 character name starts with aj.
- ii) Write database trigger to preserve the old values of student fees details before updating in table.

SPPU : Dec.-18,19, End Sem, Marks 5

Solution :

i) SQL Query is as follows -

```
SELECT name,total_fees_deposited
FROM student_fees_detail
WHERE LEN(name)>2 AND name LIKE 'aj%'
```

ii) Step 1 : First of all we will create the table **stud_fees_detail** using following SQL statement.

```
CREATE TABLE stud_fees_detail
(
  name VARCHAR2(30),
  fees_deposited NUMBER(5),
  till_date DATE
);
```

Step 2 : Insert values the values into this table using following SQL statement -

```
INSERT INTO stud_fees_detail VALUES('AAA',2000,'27-JUNE-19');
INSERT INTO stud_fees_detail VALUES('BBB',5000,'16-May-20');
INSERT INTO stud_fees_detail VALUES('CCC',1000,'01-August-19');
INSERT INTO stud_fees_detail VALUES('DDD',3000,'28-March-20');
INSERT INTO stud_fees_detail VALUES('EEE',2500,'12-November-20');
```

Step 3 : Simply create a table backup_fees using following SQL statement -

```
CREATE TABLE backup_fees
(
  name VARCHAR2(30),
  fees_deposited NUMBER(5),
  till_date DATE
);
```

Step 4 : The PL/SQL script for the updating values is as shown below

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE TRIGGER Stud_fees_AU
AFTER UPDATE ON stud_fees_detail
FOR EACH ROW
ENABLE

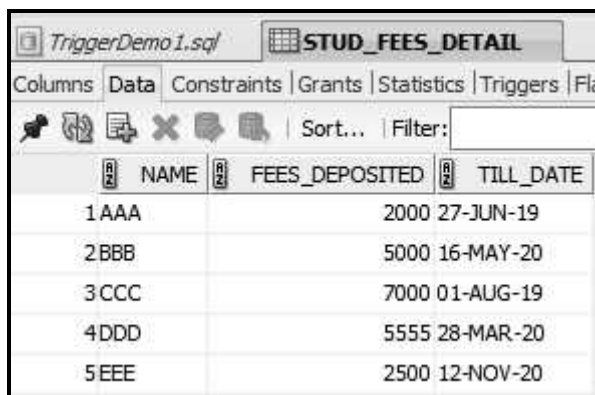
BEGIN
  INSERT INTO backup_fees
  (name,
  fees_deposited,
  till_date
  )
  VALUES
  ( :old.name,
  :old.fees_deposited,
  :old.till_date
  );
END;
/

UPDATE stud_fees_detail
SET fees_deposited=7000
WHERE name='CCC'

UPDATE stud_fees_detail
SET fees_deposited=5555
WHERE name='DDD'
```

Output

Stud_fees_detail Table(This table is updated as trigger executes)



NAME	FEES_DEPOSITED	TILL_DATE
1 AAA	2000	27-JUN-19
2 BBB	5000	16-MAY-20
3 CCC	7000	01-AUG-19
4 DDD	5555	28-MAR-20
5 EEE	2500	12-NOV-20

Back_fees Table

NAME	FEES_DEPOSITED	TILL_DATE
1CCC	1000	01-AUG-19
2DDD	3000	28-MAR-20

3.14 Assertions

- An assertion is a predicate expressing a condition we wish the database to always satisfy.
- When created, the expression must be true.
- DBMS checks the assertion after any change that may violate the expression.
- **Syntax for creating an assertion**

```
CREATE ASSERTION <assertion-name> CHECK <predicate>
```

It must return true

- **Example**

Consider following relation of customer

```
customer(customer_name, customer_street, customer_city)
```

If we want that the customer city should not be NULL then the Assertion can be written as follows

```
CREATE ASSERTION city_not_null Check
( NOT EXISTS
(Select *
  From   customer
  Where  customer_city is null));
```

Difference between Assertion and Trigger

Sr. No.	Assertion	Trigger
1.	Assertions do not modify the data. They only check certain condition.	Triggers are powerful than assertions because they can check the condition as well as they can modify the data.
2.	Assertions are not linked to specific tables in the database and not linked to specific events.	Triggers are linked to specific tables and specific events.

3.	All assertions can be implemented as triggers.	All triggers cannot be implemented as assertions.
4.	Oracle does not support assertion.	Oracle support Triggers.

3.15 Roles and Privileges

Roles

- Role is a set of privileges that can be granted to users. The roles are used for administrating the database. The privileges can be added to the role and the role can be granted to the user.
- The role can be created in Oracle using CREATE ROLE command. For example -

```
CREATE ROLE data_manager;
```

- The IDENTIFIED BY clause is used for authentication of the user. It adds the security layer to the role. For instance - In following example, the role data_manager is created and it is identified using the password 'password 1234'.

```
CREATE ROLE data_manager.  
IDENTIFIED BY password1234
```

- The privileges or permissions to particular roles can be given with the command GRANT. For example -

```
GRANT create table,create view  
TO data_manager;
```

Advantages of role management

- It is possible to assign multiple roles to the user.
- One can assign password to particular role for security purpose.
- Rather than assigning privileges at one time to some user, we can create the role and assign privileges to the role and then grant that role to multiple users.
- We can add or delete some privileges to the role and all the users are assigned with that role get those privileges or loss that privileges automatically.

Privileges

- Privileges are the access rights provided to the user on database object.
- Some of the examples of privileges are -
 - Connect to database
 - Create a table
 - Select a row from another user's table.

- There are two types of privileges -
 1. **System privilege** : This allows user to CREATE, ALTER or DROP the database objects such as table, views and so on.
 2. **Object privilege** : This allows user to SELECT, INSERT, UPDATE, or DELETE data from database objects.

3.16 Exceptions

SPPU : Dec.-18, May-19, Marks 5

Exception is an unusual situation that when occurs interrupts the normal flow of execution. The exception handling mechanism allows to handle this runtime error situation in such a way that the normal flow of execution may not get interrupted.

There are two types of exceptions in PL/SQL

- 1) **System Defined Exceptions** : These exceptions are predefined in PL/SQL which get raised WHEN certain database rule is violated. For example - Consider following Student table

RollNo	Name	Marks
101	Shilpa	55
102	Trupti	66
103	Pradnya	43
104	Sharda	42
105	Vijaya	45

We can write the exception as follows -

```
DECLARE
s_roll customers.RollNo%type := 100;
s_name customers.Name%type;
s_marks customers.Marks%type;
BEGIN
SELECT Name INTO s_name
FROM student
WHERE RollNo = s_RollNo;
DBMS_OUTPUT.PUT_LINE ('Name: ' || s_name);
EXCEPTION
WHEN no_data_found THEN
dbms_output.put_line('Student is not present!');
WHEN others THEN
dbms_output.put_line('Error!!!');
END;
/
```

After execution of above PL/SQL program we get following output
Student is not present!

Explanation : The student with roll number 100 is not present in the Student table, hence is the output.

Here NO_DATA_FOUND is a pre-defined exception which is raised when a SELECT INTO statement returns no rows.

2) User defined Exception : PL/SQL facilitates their users to define their own exceptions according to the need of the program. A user-defined exception can be raised explicitly, using either a RAISE statement or the procedure DBMS_STANDARD.RAISE_APPLICATION_ERROR.

Syntax

```
DECLARE  
my-exception EXCEPTION;
```

Example

```
DECLARE  
s_roll customers.RollNo%type := 100;  
s_name customers.Name%type;  
s_marks customers.Marks%type;  
my_exception Exception;  
BEGIN  
IF s_roll <= 0 THEN  
    RAISE my_exception;  
ELSE  
    SELECT Name INTO s_name  
FROM student  
    WHERE RollNo = s_RollNo;  
    DBMS_OUTPUT.PUT_LINE ('Name: ' || s_name);  
END IF;  
EXCEPTION  
WHEN my_exception THEN  
    dbms_output.put_line('Roll number must be greater than zero');  
WHEN no_data_found THEN  
    dbms_output.put_line('Student is not present!');  
WHEN others THEN  
    dbms_output.put_line('Error!!!');  
END;  
/
```

Example 3.16.1 Write a PL/SQL block using user defined exception for following requirements :

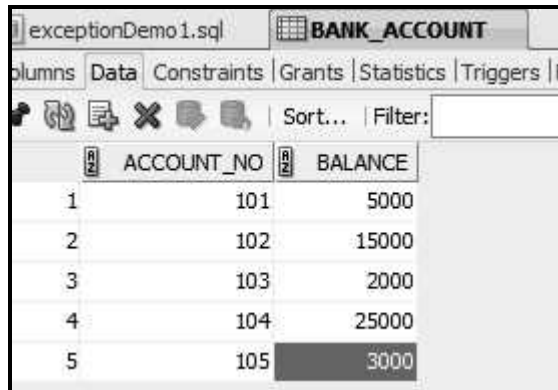
The Bank_Account table records the current balance for an account, which is updated whenever, any deposits or withdrawals takes place. If the withdrawal attempted is more than the current balance held in the account. The user defined exception is raised, displaying an appropriate message.

Note : Assume table :- Bank_Account (Account_No, Balance)

SPPU : Dec.-18, End Sem, Marks 5

Solution :

Step 1 : Table named **Bank_Account** is created as follows -



	ACCOUNT_NO	BALANCE
1	101	5000
2	102	15000
3	103	2000
4	104	25000
5	105	3000

Step 2 : Now create PL/SQL Program as follows

```
SET SERVEROUTPUT ON;
DECLARE
temp_account_no NUMBER(5);
temp_balance NUMBER(10);
withdraw_amount NUMBER(10) := 0;
Insufficient_balance EXCEPTION;
BEGIN

temp_account_no := &temp_account_no;
withdraw_amount := &withdraw_amount;
SELECT balance INTO temp_balance FROM bank_account WHERE account_no =
temp_account_no;
DBMS_OUTPUT.put_line('Account No = ' || temp_account_no || ' Balanace = ' || temp_balance);
IF(temp_balance < withdraw_amount) THEN
RAISE Insufficient_balance;
END IF;
temp_balance := temp_balance - withdraw_amount;
DBMS_OUTPUT.put_line('Current Balance is: ' || temp_balance);

EXCEPTION WHEN Insufficient_balance THEN
DBMS_OUTPUT.put_line('Insufficient Balance!!!');
END;
/
```

Output(Run 1)

```
anonymous block completed
Account No = 103 Balanace = 2000
Insufficient Balance!!!
```

Output(Run 2)

```
anonymous block completed
Account No = 104 Balanace = 25000
Current Balance is: 20000
```

Example 3.16.2 Write a PL/SQL block for following requirement and handle the exceptions.

Roll no. of student will be entered by user. Attendance of roll no. entered by user will be checked in student table. If attendance is less than 75 % then display the message "Term not granted" and set the status in student table as "D". Otherwise display message "Term granted" and set the status in student table as "ND".

SPPU : May-19, End Sem, Marks 5

Solution :

```
SET SERVEROUTPUT ON;
DECLARE
temp_rollno NUMBER(3);
temp_att NUMBER(5);
total_days NUMBER(5) := 200;
attend_ex EXCEPTION;
BEGIN
temp_rollno := &temp_rollno;
SELECT att INTO temp_att FROM stud_att WHERE roll_no = temp_rollno;
DBMS_OUTPUT.put_line('Roll No = ' || temp_rollno || ' Attendance = ' || temp_att || 'days out of 200');
IF(temp_att < (total_days*0.75)) THEN
RAISE attend_ex;
ELSE
DBMS_OUTPUT.put_line('Term Granted!!!');
UPDATE stud_att
SET status = 'ND'
WHERE roll_no = temp_rollno;
END IF;

EXCEPTION WHEN attend_ex THEN
DBMS_OUTPUT.put_line('Term not Granted!!!');
UPDATE stud_att
SET status = 'D'
WHERE roll_no = temp_rollno;
END;
/
```

Output

On subsequent execution of the above PL/SQL code following output can be obtained.

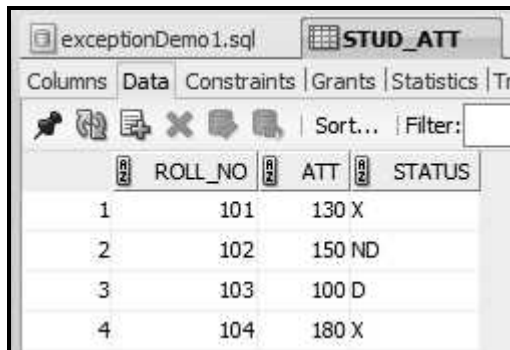
```

anonymous block completed
Roll No = 103 Attendance = 100days outof 200
Term not Granted!!!

anonymous block completed
Roll No = 102 Attendance = 150days outof 200
Term Granted!!!

```

We can verify the above execution by opening the **stud_att** table. Note that the status is updated as per the required condition.



The screenshot shows a database application window with a tab labeled 'STUD_ATT'. Below the tab are tabs for 'Columns', 'Data', 'Constraints', 'Grants', 'Statistics', and 'Triggers'. The 'Data' tab is selected, showing a table with columns 'ROLL_NO', 'ATT', and 'STATUS'. The table contains four rows of data:

	ROLL_NO	ATT	STATUS
1	101	130	X
2	102	150	ND
3	103	100	D
4	104	180	X

Example 3.16.3 Write PL/SQL code block that raise a user defined exception when business rule is violated. Business rule for client - master table specifies when the value of bal-due field is less than 0 handle the exception.

SPPU : May-19, End Sem, Marks 5

Solution :

```

SET SERVEROUTPUT ON;
DECLARE
temp_client_id NUMBER(5);
temp_balance NUMBER(10);
Insufficient_balance EXCEPTION;
BEGIN
SELECT bal_due INTO temp_balance FROM client_master WHERE bal_due < 0;
IF(temp_balance<0) THEN
RAISE Insufficient_balance;
END IF;
DBMS_OUTPUT.put_line('Success');

EXCEPTION WHEN Insufficient_balance THEN
DBMS_OUTPUT.put_line('Business Rule Violated!!!');
END;
/

```

Unit - II

Multiple Choice Questions

- Q.1** What is the full form of SQL ?
- ☐ a Standard Query Language ☐ b Structured Query Language
☐ c Simple Query Language ☐ d Specification Query Language
- Q.2** The SELECT query belongs to following category of SQL Command :
- ☐ a DDL ☐ b DML
☐ c DCL ☐ d None of these
- Q.3** The database language used for defining the schema of the database is called ____.
- ☐ a DDL ☐ b DML
☐ c DCL ☐ d None of these
- Q.4** DDL stands for ____.
- ☐ a Data Decoration Language ☐ b Data Definition Language
☐ c Dual Data Language ☐ d None of these
- Q.5** Which of the following is not a valid SQL type ?
- ☐ a Decimal ☐ b Float
☐ c Numeric ☐ d Character
- Q.6** Which operator tests column for the absence of data ?
- ☐ a EXIST operator ☐ b IS NULL operator
☐ c NOT operator ☐ d None of these
- Q.7** In SQL following command is used to change table's storage characteristics ?
- ☐ a MODIFY ☐ b CHANGE
☐ c ALTER ☐ d All of these
- Q.8** Which operator is used to compare a value to specified list of values ?
- ☐ a ANY ☐ b BETWEEN
☐ c ALL ☐ d IN
- Q.9** Which SQL keyword is used to retrieve maximum value ?
- ☐ a MOST ☐ b MAX
☐ c TOP ☐ d UPPER

Q.10 In SQL which command is used to SELECT only one copy of each set of duplicable rows _____.

- | | |
|--|--|
| ☐ a SELECT DISTINCT | ☐ b SELECT UNIQUE |
| ☐ c SELECT DEFINITE | ☐ d None of these |

Q.11 The FROM clause is used to specify _____.

- | | |
|---|--|
| ☐ a range for search condition | ☐ b actual search condition |
| ☐ c table from which data is searched. | ☐ d all of the above |

Q.12 Which of the following SQL command is used to retrieve data ?

- | | |
|------------------------------------|-----------------------------------|
| ☐ a DELETE | ☐ b INSERT |
| ☐ c RETRIVE | ☐ d SELECT |

Q.13 Which of the following command is used to add data to database table ?

- | | |
|-----------------------------------|---|
| ☐ a Add | ☐ b Insert |
| ☐ c Append | ☐ d All of the above |

Q.14 Which of the following is an aggregate function ?

- | | |
|---------------------------------|----------------------------------|
| ☐ a LEFT | ☐ b RIGHT |
| ☐ c MAX | ☐ d JOIN |

Q.15 In SQL, which command is used to issue multiple CREATE TABLE, CREATE VIEW and GRANT statements in a single transaction ?

- | | |
|---|---|
| ☐ a CREATE PACKAGE | ☐ b CREATE SCHEMA |
| ☐ c CREATE CLUSTER | ☐ d All of the above |

Q.16 The intersect operation _____.

- | |
|---|
| ☐ a automatically eliminates duplicates |
| ☐ b automatically eliminates duplicates, if we provide all clause with intersect |
| ☐ c never eliminates duplicates |
| ☐ d none of these |

Q.17 Which of the following join is also called as an inner join ?

- | | |
|--------------------------------------|--|
| ☐ a Self-join | ☐ b Non Equi-join |
| ☐ c Equi-join | ☐ d None of these |

Q.18 The virtual table created as a result of SELECT statement is called _____.

- | | |
|-------------------------------------|--|
| ☐ a view | ☐ b synonym |
| ☐ c sequence | ☐ d transaction |

Q.19 Which of the following aggregate functions ignore NULL values ?

☐ a MAX

☐ b COUNT

☐ c SUM

☐ d All of the above

Q.20 Following keyword is used with wildcards ____.

☐ a IN

☐ b BETWEEN

☐ c LIKE

☐ d UNIQUE

Q.21 What is the meaning of LIKE '%0%0%' ?

☐ a begins with two 0's

☐ b ends with two 0's

☐ c two 0's at any position

☐ d none of these

Answers Keys for Multiple Choice Questions :

Q.1	b	Q.2	b	Q.3	a	Q.4	b
Q.5	a	Q.6	b	Q.7	c	Q.8	d
Q.9	b	Q.10	a	Q.11	c	Q.12	d
Q.13	b	Q.14	c	Q.15	b	Q.16	a
Q.17	c	Q.18	a	Q.19	d	Q.20	c
Q.21	c						

